

Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет
«Высшая школа экономики»

На правах рукописи

Соколов Павел Павлович

**ПРАКТИЧНЫЕ ЗАВИСИМЫЕ ТИПЫ: ВВЕДЕНИЕ ЛЕГКОВЕСНЫХ
МЕТОДОВ ФОРМАЛЬНОЙ ВЕРИФИКАЦИИ В МЕЙНСТРИМНОЕ
ПРОГРАММИРОВАНИЕ**

ДИССЕРТАЦИЯ
на соискание ученой степени
кандидата компьютерных наук

Научный руководитель:
доктор физико-математических наук,
профессор Кузнецов Степан Львович

Москва — 2027

Оглавление

Введение	4
Актуальность исследования	4
Степень разработанности исследовательской проблемы	4
Объект и предмет исследования	5
Цель и задачи исследования	6
Методология и методы исследования	6
Научная новизна исследования	6
Практическая значимость работы	7
Степень достоверности и апробация результатов	7
1 Чистая система типов с рангами функций в качестве сортов	8
1.1 Мотивация и идея	8
1.2 Синтаксис чистых систем типов	8
1.3 ЧСТ как система подстановок	10
1.3.1 Общие факты об отношениях	10
1.3.2 Конгруэнции	12
1.3.3 Отношение одношаговой редукции	13
1.3.4 Конфлюэнтность редукций	14
1.3.5 Вычислительное равенство термов	14
1.4 Типизация ЧСТ	15
1.4.1 Контекст	15
1.4.2 Сорт	16
1.4.3 Правила типизации	16
1.4.4 Леммы об инверсии	17
1.4.5 Сохранение типизации	18
1.5 О здравости чистых систем типов	18
1.5.1 Сильная нормализация	18
1.5.2 Теоретико-множественная семантика	20
1.6 О здравости чистой системы типов, основанной на рангах	20
1.6.1 Сильная нормализация	21
1.6.2 Теоретико-множественная семантика	22
2 К системам типов, способных к самореинтерпретации	24
2.1 Введение	24
2.1.1 Вклад	25
2.2 Теорема о невозможности внутренней интерпретации	25
2.3 Почти просто типизированный почти рефлексивный язык: НМ-R	27
2.3.1 Метатеоретические свойства НМ-R	31

2.4	Почти рефлексивный язык с зависимыми типами: MLTT-R	33
2.4.1	Правила вычисления MLTT-R и модели систем с зависимыми типами .	36
2.4.2	Метатеоретические свойства MLTT-R	41
2.4.3	Полезные реинтерпретации внутри MLTT-R	42
2.5	Обсуждение и связанные работы	43
2.5.1	Что было сделано до нас	43
2.5.2	Что можно было сделать иначе	44
2.6	Заключение и дальнейшая работа	46
Заключение		47
	Основные результаты исследования	47
	Общие выводы	47
	Перспективы дальнейшей разработки	47
	Основные публикации соискателя	47
A	Список сокращений и условных обозначений	48
B	Словарь терминов	49
C	Список литературы	50
D	Список иллюстрированного материала	53
E	Приложения	54

Введение

Актуальность исследования

Подавляющее большинство используемых сегодня языков программирования не подходит для написания корректного по построению кода. В случае же, когда требуется удостовериться в корректном исполнении программы (исполнение без ошибок, соответствие спецификации и так далее), программисту необходимо настраивать дополнительные инструменты: статические анализаторы, автоматическое тестирование, средства проверки модели и так далее. К сожалению, ни одно из используемых средств не решает задачу полностью: какие-то из них не гарантируют корректности¹, другие имеют (сверх)экспоненциальные асимптотики исполнения и не могут быть использованы для полного анализа проверяемых систем². В конце концов, сама необходимость дополнительной настройки, коммерческое лицензирование и сложность использования мешают распространению подобных технологий.

Так, исчерпывающим решением было бы легковесное, бесплатное, удобное для использования, встроенное в стандартную поставку языка программирования средство формальной спецификации и формальной верификации программной системы; по определению, сформулированному Бенджамином К. Пирсом [38], таким средством является *система типов*. Приведём его здесь:

Система типов — это гибко управляемый синтаксический метод доказательства отсутствия в программе определенных видов поведения при помощи классификации выражений языка по разновидностям вычисляемых ими значений (*типам*).

Становится возможным поставить следующую *исследовательскую проблему*: Какой должна быть система типов языка программирования, чтобы

- Она позволяла легко формулировать и легко (в идеале — автоматически) доказывать свойства программ, интересные в мейнстримном программировании?
- Полученным языком было удобно пользоваться широкому кругу пользователей?

Степень разработанности исследовательской проблемы

К подтверждению гипотезы, что теорию типов можно использовать для формальной верификации компьютерных программ, логики и исследователи компьютерных наук приходили постепенно. Пожалуй, первым удачным экспериментом в этой области следует считать Исчисление Конструкций (Calculus of Constructions, CoC) [14] Тьерри Коканда, в

¹Здесь уместно процитировать Эдсгера В. Дейкстру: “... тестирование программы может весьма эффективно продемонстрировать наличие багов, но совершенно неприменимо, чтобы показать их отсутствие” [15].

²Однако, например, проверку моделей вполне успешно используют для верификации критических компонентов системы и рискованных оптимизаций. [28]

котором можно не только реализовывать алгоритмы в виде программ на некотором функциональном языке, но и доказывать их свойства в той же самой системе; практическая реализация этого исчисления сегодня известна как Rosq (ранее Coq) [34].

Эта замечательная особенность CoC возникла в результате принятия парадигмы *утверждения в качестве (некоторых) типов* [37]. В её рамках доказываемое утверждение φ кодируется в виде некоторого *типа* F , представимого в выбранной системе (так что язык типов должен быть достаточно выразительным, чтобы кодировать все утверждения, которые мы хотим доказать). Тогда задача доказательства $\vdash \varphi$ сводится к задаче построения выражения f , имеющего тип F ($\vdash f : F$); поиск такого выражения может производиться как вручную, так и с помощью автоматических средств³.

Однако следует иметь в виду, что Rosq и другие системы интерактивных доказательств в первую очередь являются *системами доказательств*, нежели *языками программирования*: как правило, скрипты для таких систем не запускаются в качестве самостоятельных программ, а только подаются на вход алгоритму проверки типов⁴. Значимыми исключениями являются языки Agda и Lean: в соответствующих компиляторах есть опции, позволяющие компилировать и запускать скрипты на этих языках.

Agda [29], как и язык, на котором она реализована (Haskell), в первую очередь является тестовой площадкой для различных расширений CoC: доступны опции для подключения коиндуктивных типов [3] и ограниченной рекурсии [13], кубических типов [36], логически противоречивого правила Type-in-Type и многих других. К сожалению, это делает ядро системы перегруженным и малоприменимым в критических ситуациях.

Система Lean [26] получила широкую известность ввиду прорывного по тогдашним меркам инструментария и распространения в среде математиков-энтузиастов. Кроме этого, архитектура системы типов и стандартных библиотек была спроектирована с учётом удобства переноса доказательств из классической математики; в решении, избранном создателями Lean, для этого пришлось пожертвовать достаточно важными свойствами базовой системы CoC, в частности свойством *сохранения типизации*: если $e_1 : T$ и $e_1 \rightarrow e_2$ (« e_1 за один шаг вычисляется в e_2 »), то $e_2 : T$. Повторюсь: это свойство в Lean НЕ выполняется. Если быть точным, можно подобрать типизируемый терм e_1 такой, что результат вычисления e_2 не будет типизироваться. Данное обстоятельство, конечно, не делает данную логическую систему противоречивой (в частности, существует теоретико-модельное доказательство непротиворечивости системы Lean [10]), но является серьёзным неудобством при неумелой работе с системой. Более того, если такое базовое синтаксическое свойство не выполнено, возникают сомнения в применимости подобной системы для программирования, т.е. акта дизайна программ для исполнения на переписывающей системе.

Таким образом, можно сделать вывод, что на данный момент не существует *языка программирования* на основе CoC, сохраняющего важные вычислительные свойства и пригодного для разработки производительных программных систем целиком в этом языке.

Объект и предмет исследования

Объектом данного исследования является система типов CoC и её расширения, а также практическая реализация эффективной среды исполнения CoC.

Предметом исследования являются такие аспекты CoC и её расширений, которые сделают её пригодной для промышленного применения в качестве системы исполнения высокопроизводительных программ, корректных по построению.

³В последнее время стала популярной методика, по которой доказательство математических утверждений ищут с помощью больших языковых моделей, генерирующих программы в одной из таких систем. [33]

⁴В Rosq, однако, есть обходной механизм *извлечения доказательства*, транслирующий скрипт Rosq в программу на OCaml/Haskell/Scheme/...

Цель и задачи исследования

Целью данной работы является поиск системы типов, основанной на CoC, пригодной для написания корректных по построению программ и доступной широкому кругу специалистов в сфере информационных технологий, а также её эффективная реализация.

Для достижения этой цели необходимо выполнить следующие *задачи*:

1. Исследование возможных (как существующих, так и новых) расширений системы CoC;
2. Отбор наиболее необходимых для мейнстримного программирования расширений;
3. Доказательство сохранения важных вычислительных свойств в расширенной системе;
4. Дизайн эргономичного алгоритма вывода типов, поддерживающего существующие и будущие паттерны программирования с зависимыми типами;
5. Реализация эффективной среды исполнения и логического ядра системы;
6. Создание демонстрационных проектов на новом языке, показывающих состоятельность выбранного подхода.

Методология и методы исследования

В силу природы поставленных задач, настоящее исследование производится на стыке математической логики (теория типов, теория моделей), алгебры (символьные вычисления, алгебраическая оптимизация) и теоретической информатики (теория алгоритмов, теория сложности) с акцентом на практической реализации. Для каждой из поставленных задач используются свои методы исследования, в частности:

1. Полученная система типов описывается в декларативной форме, в виде генценовского исчисления секвенций специального вида;
2. Метатеоретические свойства системы доказываются относительно сформулированного исчисления структурной индукцией по дереву естественного вывода в системе;
3. Алгоритм вывода типов также формулируется с помощью исчисления секвенций, но уже в алгоритмической форме, демонстрируется корректность вывода относительно базовой системы;
4. Реализация компилятора и среды исполнения опирается на новейшие практики реализации подобных программных систем;
5. Эффективность среды исполнения проверяется посредством бенчмаркинга и сравнения производительности по сравнению с аналогичными средами на эталонных программах, реализованных на различных языках.

Научная новизна исследования

Данное исследование предлагает новые дизайны для систем типов с зависимыми типами с акцентом на практическую применимость в промышленном программировании, основываясь на новейших результатах в области оснований математики; кроме этого, настоящая работа содержит значимые продвижения в техниках создания эффективных сред исполнения для языков с зависимыми типами, а также в техниках использования зависимых типов в промышленном программировании.

Практическая значимость работы

Существование удобного в использовании языка программирования с эффективной средой исполнения, позволяющего создавать корректные по построению программы, качественно изменит подход к разработке программного обеспечения. На подобные программные решения существует большой спрос, что показывает пример Rust [2]; технология, позволяющая раздвинуть границы возможного ещё дальше, будет востребована не меньше.

Степень достоверности и апробация результатов

В лучших традициях дисциплины *теории типов*, достоверность ключевых теоретических результатов, полученных в ходе исследования, гарантируется формализацией доказательств в среде интерактивных доказательств Rosq. Апробация практических результатов произведена с помощью реализации ряда демонстрационных проектов в полученном языке программирования, показывающих состоятельность и перспективность выбранного подхода.

1. Чистая система типов с рангами функций в качестве сортов

1.1 Мотивация и идея

Больше всего вопросов при изучении современных средств интерактивных доказательств (Lean, Rocq, Agda...) вызывает устройство *иерархий вселенных*: $\text{Type}_1 : \text{Type}_2 : \dots$. Вызывают они вопросы и с метатеоретической точки зрения — в доказательствах здравости Type_n интерпретируется как n -й недостижимый кардинал.

Поэтому интересно искать альтернативные варианты устройства иерархии. Далее описан забавный вариант, располагающий типы по “вселенным” соответственно *рангу функций*. Термы в этой системе сильно нормализуются, так что она может быть использована в качестве основы для средства автоматической проверки доказательств. Также нами была построена теоретико-множественная модель этой системы в ZFC, открывающая возможные применения нашей системы в высшей теории вычислимости.

Большая часть описанных результатов формально верифицирована на Rocq.

1.2 Синтаксис чистых систем типов

Наша система является конкретным экземпляром *Чистой системы типов*. ЧСТ определяется поверх множества *термов*, параметризованного множеством переменных X и множеством сортов L :

Переменная	x	$\in$	X	
Сорт	ℓ	$\in$	L	
Терм	T, U, t, u	$::=$	x	переменная
			ℓ	сорт как тип
			$\Pi(x : T).U$	тип зависимых функций
			$\lambda(x : T).u$	анонимная функция
			$u \$ t$	применение функции

В формализации синтаксиса с операторами, связывающими переменные, такими как Π и λ , есть много тонкостей, зависящих от выбора представления. В нашей формализации мы выбрали представление синтаксиса как *относительной монады* — тип термов параметризован количеством свободных переменных; применение операторов связывания увеличивает количество свободных переменных в подтерме на 1; переменные представляют собой значения типа *ограниченных натуральных чисел*.

Определение 1 (Ограниченные натуральные числа). Тип $\text{Fin}(n)$ состоит из натуральных чисел, строго меньших n , с конструкторами $\text{fzero} : \text{Fin}(n + 1)$ (первый элемент) и $\text{fsucc} : \text{Fin}(n) \rightarrow \text{Fin}(n + 1)$ (преемник элемента).

Наш выбор представления мотивирован тем, что, во-первых, в реализации избегающей связывания подстановки для такой системы ошибиться гораздо сложнее, чем для системы с наивным строковым представлением переменных; во-вторых, наличие такого параметра увеличивает количество информации, доступной для элаборации движком *Rosq*, что уменьшает количество аннотаций в формулировках теорем и в доказательствах по сравнению с непараметризованными представлениями; в-третьих, параметризация натуральным числом вместо параметризации типом переменных позволяет естественным образом связать термы и контекст в правиле типизации.

Далее описаны определения синтаксических операций над термами, вспомогательных функций и связанных с ними лемм, иллюстрирующих корректность определений.

Определение 2 (Ослабление функции переименования). Для функции переименования $f : \text{Fin}(m) \rightarrow \text{Fin}(n)$ ослабление $\text{weak}(f) : \text{Fin}(m + 1) \rightarrow \text{Fin}(n + 1)$ отображает новый индекс fzero в fzero и сдвигает остальные согласно f .

Определение 3 (Переименование). Для функции переименования $f : \text{Fin}(m) \rightarrow \text{Fin}(n)$ операция переименования **rename** преобразует терм в контексте длины m в терм в контексте длины n : вселенные остаются без изменений; в типе зависимых функций область определения переименовывается по f , а область значений — по $\text{weak}(f)$; индексы переменных отображаются по f ; в лямбде область определения переименовывается по f , а тело — по $\text{weak}(f)$; в применении обе части переименовываются по f .

Определение 4 (Сдвиг). Сдвиг **shift** терма t в контексте длины n получает терм в контексте длины $n + 1$ путём увеличения каждого индекса переменной (т. е. переименование по функции, отображающей каждый индекс в его преемник).

Определение 5 (Транспонирование). Для функции подстановки $f : \text{Fin}(m) \rightarrow \text{Term}(L, n)$ транспонирование $\text{transpose}(f) : \text{Fin}(m + 1) \rightarrow \text{Term}(L, n + 1)$ по индексу i возвращает терм в контексте длины $n + 1$: при $i = \text{fzero}$ это переменная $\text{var}(\text{fzero})$; при $i = \text{fsucc}(j)$ это $\text{shift}(f(j))$.

Определение 6 (Подстановка). Для функции подстановки $f : \text{Fin}(m) \rightarrow \text{Term}(L, n)$ и терма t в контексте длины m операция подстановки **replace**(f, t) заменяет переменные в t соответствующими термами из f : вселенные остаются без изменений; в типе зависимых функций подстановка в область определения происходит по f , а в область значений — по транспонированию f ; переменная i заменяется на $f(i)$; в лямбде подстановка в область определения происходит по f , а в тело — по транспонированию f ; в применении в обе части подставляется f .

Определение 7 (Подстановка в свежую переменную). Подстановка терма $v : \text{Term}(L, n)$ в свежую переменную терма $t : \text{Term}(L, n + 1)$ (обозначение в *Rosq*: $t \triangleleft v$) есть операция замены переменной с индексом fzero на v и одновременного сдвига остальных переменных на -1 .

В неформальном описании в тексте подстановка в свежую переменную x обозначается как $t[x := v]$ для читаемости.

Определение 8 (Толкание). Для терма $t : \text{Term}(L, n)$ операция толкания строит функцию подстановки $\text{push}(t) : \text{Fin}(m + n + 1) \rightarrow \text{Term}(L, m + n)$, где первые m переменных остаются без изменений, $(m + 1)$ -я отображается в t , а оставшиеся n переменных сдвигаются на -1 .

Определение 9 (Индексная подстановка). *Подстановка терма v в переменную с заданным индексом в терме t (обозначение $t \triangleleft^i v$) выполняется путём толкания v к нужному индексу и последующего применения полученной подстановки.*

Лемма 1 (Экстенциональность переименования). *Если две функции переименования f и g совпадают на всех индексах, то они дают одинаковый результат при применении к любому терму.*

Лемма 2 (Экстенциональность транспонирования). *Если две функции f и g совпадают на всех индексах, то их транспонирования совпадают на всех индексах.*

Лемма 3 (Экстенциональность подстановки). *Если две функции подстановки f и g совпадают на всех индексах, то они дают одинаковый результат при применении к любому терму.*

Лемма 4 (Композиция переименований). *Переименование по f после переименования по g равносильно переименованию по композиции $f \circ g$.*

Лемма 5 (Переименование коммутует с подстановкой). *Переименование по f терма, к которому применялась подстановка g , эквивалентно подстановке g , переименованной по f .*

Лемма 6 (Подстановка коммутует с переименованием). *Подстановка в переименованный терм равносильна подстановке по композиции соответствующих функций.*

Лемма 7 (Подстановка с тождественным отображением). *Замена переменных на самих себя есть тождественная операция на термах.*

Лемма 8 (Композиция подстановок). *Подстановка одной подстановки в другую равносильна подстановке по композиции функций.*

Лемма 9 (Переименование распределяется над подстановкой). *Переименование подстановки $t \triangleleft u$ по f совпадает с подстановкой $\text{weak}(f)(t) \triangleleft f(u)$.*

Лемма 10 (Подстановка распределяется над подстановкой). *Подстановка в подстановку $t \triangleleft u$ по f совпадает с подстановкой $\text{transpose}(f)(t) \triangleleft f(u)$.*

Лемма 11 (Подстановка в сдвинутый терм). *Подстановка в свежую переменную сдвинутого терма восстанавливает исходный терм: $\text{shift}(t) \triangleleft u = t$.*

Лемма 12 (Сдвиг коммутует с индексной подстановкой). *Сдвиг, за которым следует индексная подстановка на $m+1$, равносильна индексной подстановке на m , за которой следует сдвиг.*

1.3 ЧСТ как система подстановок

1.3.1 Общие факты об отношениях

Вспомним некоторые базовые определения из теории подстановочных систем.

Определение 10 (Рефлексивно-транзитивное замыкание). *Для множества A и отношения R на A рефлексивно-транзитивное замыкание $\text{RTC}(R)$ есть наименьшее отношение на A , генерируемое двумя конструкторами: конструктор рефлексивности, дающий $x \text{ RTC } x$ для каждого $x \in A$; и конструктор шага, который, по заданным $x R y$ и $y \text{ RTC } (R) z$, даёт $x \text{ RTC } (R) z$.*

Определение 11 (Симметрическое замыкание). Симметрическое замыкание $\text{Sym}(R)$ есть наименьшее отношение на A , генерируемое: конструктором **forward**, который по заданному $x R y$ даёт $x \text{Sym}(R) y$; и конструктором **backward**, который по заданному $y R x$ даёт $x \text{Sym}(R) y$.

Определение 12 (Обратное отношение). Обратное отношение R^{-1} определяется так: $x R^{-1} y$ тогда и только тогда, когда $y R x$.

Определение 13 (Замыкание до эквивалентности). Замыкание отношения R до эквивалентности есть рефлексивно-транзитивное замыкание симметрического замыкания: $\text{EquivClosure } R \triangleq \text{RTC}(\text{Sym } R)$.

Определение 14 (Прообраз отношения). Для функции $f : A \rightarrow B$ и отношения R на B прообраз отношения R по f (обозначение $R @ f$) есть отношение на A , определённое тем, что $x (R @ f) y$ тогда и только тогда, когда $f(x) R f(y)$.

Определение 15 (Свойство Чёрча–Россера). Отношение R на A обладает свойством Чёрча–Россера, если для всех $t, t_1, t_2 \in A$ таких, что $t R t_1$ и $t R t_2$, существует t^* такой, что $t_1 R t^*$ и $t_2 R t^*$:

$$\begin{array}{ccc} t & \longrightarrow & t_1 \\ \downarrow & & \downarrow \\ t_2 & \dashrightarrow & t^* \end{array}$$

Далее, словом **Экземпляр** обозначены факты, которые были сохранены нами в базе экземпляров Rosq для автоматического вывода свойств конкретных отношений, введённых нами в ходе формализации.

Экземпляр 1. Рефлексивно-транзитивное замыкание любого отношения рефлексивно.

Экземпляр 2. Рефлексивно-транзитивное замыкание любого отношения транзитивно.

Лемма 13. Если $x R y$, то $x \text{RTC}(R) y$.

Экземпляр 3. Если R симметрично, то его рефлексивно-транзитивное замыкание также симметрично.

Экземпляр 4. Симметрическое замыкание любого отношения симметрично.

Экземпляр 5. Замыкание любого отношения до эквивалентности есть отношение эквивалентности (т. е. является рефлексивным, симметричным и транзитивным).

Лемма 14. Если $x \text{RTC}(R^{-1}) y$, то $y \text{RTC}(R) x$.

Лемма 15 (Отображение через рефлексивно-транзитивное замыкание). Для функции $f : A \rightarrow B$ и отношения R на B : если $x \text{RTC}(R @ f) y$, то $f(x) \text{RTC}(R) f(y)$.

Лемма 16 (Монотонность рефлексивно-транзитивного замыкания). Если $Q \subseteq R$ (т. е. $Q x y \Rightarrow R x y$ для всех x, y), то $\text{RTC } Q \subseteq \text{RTC } R$.

Лемма 17 (Отображение через симметрическое замыкание). Для функции $f : A \rightarrow B$ и отношения R на B : если $x \text{Sym}(R @ f) y$, то $f(x) \text{Sym}(R) f(y)$.

Лемма 18 (Монотонность симметрического замыкания). Если $Q \subseteq R$, то $\text{Sym } Q \subseteq \text{Sym } R$.

Лемма 19. Если $x R y$, то $x \text{EquivClosure}(R) y$.

Лемма 20 (Отображение через замыкание до эквивалентности). Для функции $f : A \rightarrow B$ и отношения R на B : если $x \text{ EquivClosure } (R @ f) y$, то $f(x) \text{ EquivClosure } (R) f(y)$.

Лемма 21 (Монотонность замыкания до эквивалентности). Если $Q \subseteq R$, то $\text{EquivClosure } (Q) \subseteq \text{EquivClosure } (R)$.

Лемма 22 (Экстенциональность свойства Чёрча–Россера). Если два отношения Q и R на A равны (т. е. $Q(a, b) \Leftrightarrow R(a, b)$ для всех a, b), то Q обладает свойством Чёрча–Россера тогда и только тогда, когда R обладает свойством Чёрча–Россера.

Теорема 1 (Свойство Чёрча–Россера для рефлексивно-транзитивного замыкания). Если отношение R обладает свойством Чёрча–Россера, то его рефлексивно-транзитивное замыкание $\text{RTC } R$ также обладает свойством Чёрча–Россера.

1.3.2 Конгруэнции

В этом разделе мы определим, что такое конгруэнция на термах. Здесь и далее, словом **Класс** мы будем обозначать определения, соответствие которым удобно считать **Экземплярами** для автоматического вывода свойств в Rosq.

Класс 1 (Конгруэнция). Семейство отношений R на термах (индексируемое длиной контекста) является конгруэнцией, если для него выполнен набор из шести условий совместимости:

- отношение совместимо с типом функций по области определения (если $T R T'$, то $(\Pi(x : T).U) R (\Pi(x : T').U)$) и по области значений (если $U R U'$, то $(\Pi(x : T).U) R (\Pi(x : T).U')$);
- отношение совместимо с лямбдами по области определения (если $T R T'$, то $(\lambda(x : T).t) R (\lambda(x : T').t)$) и по телу (если $t R t'$, то $(\lambda(x : T).t) R (\lambda(x : T).t')$);
- отношение совместимо с аппликациями по функции (если $t R t'$, то $(t \$ u) R (t' \$ u)$) и по аргументу (если $u R u'$, то $(t \$ u) R (t \$ u')$).

Экземпляр 6 (Симметрическое замыкание сохраняет конгруэнтность). Если R — конгруэнция, то семейство симметрических замыканий $\text{Sym } (R)$ также является конгруэнцией.

Экземпляр 7 (Рефлексивно-транзитивное замыкание сохраняет конгруэнтность). Если R — конгруэнция, то семейство рефлексивно-транзитивных замыканий $\text{RTC } (R)$ также является конгруэнцией.

Лемма 23 (Параллельная конгруэнтность для типов функций). Если R — транзитивная конгруэнция, а $T R T'$ и $U R U'$, то $(\Pi(x : T).U) R (\Pi(x : T').U')$.

Лемма 24 (Параллельная конгруэнтность для лямбд). Если R — транзитивная конгруэнция, а $T R T'$ и $t R t'$, то $(\lambda(x : T).t) R (\lambda(x : T').t')$.

Лемма 25 (Параллельная конгруэнтность для аппликаций). Если R — транзитивная конгруэнция, а $t R t'$ и $v R v'$, то $(t \$ v) R (t' \$ v')$.

1.3.3 Отношение одношаговой редукции

Наконец, мы готовы вернуться к описанию чистой системы типов. Следующий шаг — определение редукций, т.е. вычислений в системе. Одношаговая редукция термов определяется как наименьшее конгруэнтное отношение, выполняющее правило бета-редукции о применении анонимной функции к аргументу:

$$\frac{T \rightarrow_{\beta} T'}{\Pi(x : T).U \rightarrow_{\beta} \Pi(x : T').U} \quad \frac{U \rightarrow_{\beta} U'}{\Pi(x : T).U \rightarrow_{\beta} \Pi(x : T).U'} \quad \frac{T \rightarrow_{\beta} T'}{\lambda(x : T).t \rightarrow_{\beta} \lambda(x : T').t}$$

$$\frac{t \rightarrow_{\beta} t'}{\lambda(x : T).t \rightarrow_{\beta} \lambda(x : T).t'} \quad \frac{t \rightarrow_{\beta} t'}{t \$ u \rightarrow_{\beta} t' \$ u} \quad \frac{u \rightarrow_{\beta} u'}{t \$ u \rightarrow_{\beta} t \$ u'} \quad (\lambda(x : T).t) \$ u \rightarrow_{\beta} t[x := u]$$

Ниже — несколько лемм, показывающих взаимоотношения между редукцией и уже определёнными операциями на термах.

Экземпляр 8. *Одношаговая β -редукция является конгруэнцией.*

Определение 16 (Многошаговая β -редукция). *Отношение $\xrightarrow{*}_{\beta}$ есть рефлексивно-транзитивное замыкание отношения $\xrightarrow{\beta}$.*

Лемма 26 (Переименование коммутрует с шагом). *Если $t \xrightarrow{\beta} t'$, то $\text{rename}(f)(t) \xrightarrow{\beta} \text{rename}(f)(t')$.*

Лемма 27 (Подстановка коммутует с шагом). *Если $t \xrightarrow{\beta} t'$, то $\text{replace}(f)(t) \xrightarrow{\beta} \text{replace}(f)(t')$.*

Лемма 28 (Лямбда-инверсия для переименования). *Если переименование терма совпадает с некоторой анонимной функцией, то исходный терм сам является анонимной функцией, и переименование раскладывается соответственно.*

Лемма 29 (Инверсия шага для переименования). *Если $\text{rename}(f)(t) \xrightarrow{\beta} u$, то существует v такой, что $u = \text{rename}(f)(v)$ и $t \xrightarrow{\beta} v$.*

Лемма 30 (Переименование коммутует с многошаговой редукцией). *Если $t \xrightarrow{*}_{\beta} t'$, то $\text{rename}(f)(t) \xrightarrow{*}_{\beta} \text{rename}(f)(t')$.*

Лемма 31 (Транспонирование коммутует с многошаговой редукцией). *Если каждый компонент функции подстановки преобразуется в соответствующий компонент другой, то их транспонирования также преобразуются друг в друга.*

Лемма 32 (Подстановка коммутует с многошаговой редукцией). *Если каждый компонент функции подстановки преобразуется в соответствующий компонент другой, то подставленные термы также преобразуются друг в друга.*

Лемма 33 (Представление произведения). *Если $\Pi(x : T).U \xrightarrow{*}_{\beta} t$, то t является типом зависимых функций.*

Лемма 34 (Инверсия произведения для многошаговой редукции). *Если $\Pi(x : T).U \xrightarrow{*}_{\beta} \Pi(x : T').U'$, то $T \xrightarrow{*}_{\beta} T'$ и $U \xrightarrow{*}_{\beta} U'$.*

1.3.4 Конфлюэнтность редукций

Классическим результатом в теории ЧСТ является конфлюэнтность редукций, т.е. выполнение свойства Чёрча-Россера для многошаговой редукции. В этой секции мы воспроизводим это доказательство на основе доказательства для нетипизированного лямбда-исчисления [31]. Ключевой элемент доказательства — дополнительное построение *параллельной β -редукции*, производящей все доступные в терме подстановки “одновременно” и обладающей свойством Чёрча-Россера.

Определение 17 (Параллельная β -редукция).

$$\begin{array}{c} \overline{t \rightarrow_{\beta} t} \quad \frac{T \rightarrow_{\beta} T' \quad U \rightarrow_{\beta} U'}{\Pi(x : T).U \rightarrow_{\beta} \Pi(x : T').U'} \quad \frac{T \rightarrow_{\beta} T' \quad t \rightarrow_{\beta} t'}{\lambda(x : T).t \rightarrow_{\beta} \lambda(x : T').t'} \\[10pt] \frac{t \rightarrow_{\beta} t' \quad u \rightarrow_{\beta} u'}{t \$ u \rightarrow_{\beta} t' \$ u'} \quad \frac{t \rightarrow_{\beta} t' \quad u \rightarrow_{\beta} u'}{(\lambda(x : T).t) \$ u \rightarrow_{\beta} t' [x := u']} \end{array}$$

Лемма 35. Если $t \xrightarrow{\beta} t'$, то $t \rightarrow_{\beta} t'$.

Лемма 36. Если $t \rightarrow_{\beta} t'$, то $t \xrightarrow{*}_{\beta} t'$.

Лемма 37. Если $t \rightarrow_{\beta} t'$, то $\text{rename}(f)(t) \rightarrow_{\beta} \text{rename}(f)(t')$.

Лемма 38 (Транспонирование коммутует с параллельным шагом). Если каждый компонент функции подстановки делает параллельный шаг в соответствующий компонент другой, то их транспонирования также делают параллельный шаг.

Лемма 39 (Подстановка коммутует с параллельным шагом). Если каждый компонент функции подстановки делает параллельный шаг в соответствующий компонент другой, и $t \rightarrow_{\beta} t'$, то подставленные термы также делают параллельный шаг друг в друга.

Лемма 40. Если $t \rightarrow_{\beta} t'$, то $f \triangleleft t \rightarrow_{\beta} f \triangleleft t'$.

Лемма 41. Если $f \rightarrow_{\beta} f'$ и $t \rightarrow_{\beta} t'$, то $f \triangleleft t \rightarrow_{\beta} f' \triangleleft t'$.

Лемма 42 (Лямбда-инверсия для параллельного шага). Если анонимная функция делает параллельный шаг в некоторый терм t' , то t' является лямбдой, и параллельный шаг раскладывается на параллельные шаги в области определения и теле лямбды.

Лемма 43 (Конфлюэнтность параллельной редукции). Отношение параллельной β -редукции обладает свойством Чёрча-Россера.

Теорема 2 (Конфлюэнтность многошаговой редукции). Отношение многошаговой β -редукции обладает свойством Чёрча-Россера.

1.3.5 Вычислительное равенство термов

Ключевым понятием в определении правил типизации ЧСТ является *вычислительное равенство* термов.

Определение 18 (Вычислительное равенство). Два терма t и t' равны вычислительно (обозначение $t =_{\beta} t'$), если они связаны замыканием одношагового β -преобразования до эквивалентности.

Естественным образом, оно уважается базовыми синтаксическими операциями.

Лемма 44. Если $t =_{\beta} t'$, то $\text{rename}(f)(t) =_{\beta} \text{rename}(f)(t')$.

Лемма 45. Если $t =_{\beta} t'$, то $\text{replace}(f)(t) =_{\beta} \text{replace}(f)(t')$.

Кроме этого, благодаря конfluenceности системы, вычислительное равенство обладает рядом важных свойств.

Теорема 3 (Характеризация вычислительного равенства). Два терма равны вычислительно тогда и только тогда, когда существует их общий результат редукции: т. е. $t =_{\beta} t'$ тогда и только тогда, когда существует терм t^* такой, что $t \xrightarrow{*}_{\beta} t^*$ и $t' \xrightarrow{*}_{\beta} t^*$.

Лемма 46 (Инверсия типа функций для вычислительного равенства). Если $\Pi(x : T).U =_{\beta} \Pi(x : T').U'$, то $T =_{\beta} T'$ и $U =_{\beta} U'$.

Лемма 47 (Подстановка сохраняет вычислительное равенство). Если $t =_{\beta} t'$, то $f \triangleleft t =_{\beta} f \triangleleft t'$.

1.4 Типизация ЧСТ

1.4.1 Контекст

Типизация языков с операторами связывания переменных не обходится без *контекста* — структуры данных, хранящей тип каждой доступной переменной. В случае чистых систем типов, множество типов и термов совпадает, так что это просто список термов, выступающих в роли типа; тонкость в том, что каждый следующий тип в контексте может зависеть от всех ранее введённых переменных.

Определение 19 (Контексты). Контекст длины n — это либо пустой контекст $\cdot$ (длины 0), либо расширение $\Gamma, x : T$ контекста Γ длины n типом (термом) T в контексте длины n , дающее контекст длины $n + 1$.

Определение 20 (Предшественник контекста). Для непустого контекста $\Gamma = \Gamma_0, x : T$ предшественник есть Γ_0 .

Определение 21 (Вершина контекста). Для непустого контекста $\Gamma = \Gamma_0, x : T$ вершина (или головной элемент) есть T .

Определение 22 (Индексирование контекста). Для контекста Γ длины n и индекса $i \in \text{Fin}(n)$ (обозначение: $\Gamma !! i$) извлекает i -й тип из контекста (считая с вершины), с надлежащим сдвигом.

В неформальном изложении далее индексы отождествляются с переменными, так что допустимо обозначение $\Gamma !! x$.

Определение 23 (Переименователь ослабления). Функция $\text{wr} : \text{Fin}(m + n) \rightarrow \text{Fin}(m + n + 1)$, сдвигающая индексы с учётом нового типа, вставленного на позицию m , считая с вершины контекста.

Определение 24 (Вставка в контекст). Вставка типа $T : \text{Term}(L, n)$ в контекст Γ длины $m + n$ (на позицию m) даёт контекст длины $m + n + 1$ с надлежащим переименованием существующих типов.

Определение 25 (Сжатие контекста). Удаление первых m типов из контекста длины $m + n$, оставляя контекст длины n (*shrink*).

Определение 26 (Индексная подстановка в контекст). *Подстановка терма t в переменную с заданной позицией в контексте, дающая новый контекст, в котором эта переменная заменена на t с надлежащим индексом.*

Определение 27 (Сжатие до одного элемента). *Извлечение (squeeze) типа $(m + 1)$ -й переменной из контекста длины $m + n + 1$ путём снятия первых m типов.*

Лемма 48. *Применение переименователя ослабления к преемнику индекса равно преемнику применения переименователя ослабления к индексу.*

Лемма 49. *Ослабление переименователя ослабления равно самому переименователю ослабления на преемниках индексов.*

Лемма 50. *Переименование по ослаблению переименователя ослабления равносильно переименованию по переименователю ослабления.*

Лемма 51 (Преимник индекса). *Индексирование контекста, расширенного новым типом, по преемнику индекса возвращает сдвинутый результат индексирования индексом в исходном контексте.*

Лемма 52 (Вставка и индексирование). *Индексирование вставленного контекста по ослабленному индексу даёт переименование исходного индекса.*

1.4.2 Сорт

Последний и самый важный компонент, необходимый для введения правил типизации — сигнатура сортов, задающая взаимное отношение сортов друг между другом и отнесение типов зависимых функций к тем или иным сортам.

Класс 2 (Сигнатура сортов). *Сигнатура сортов на типе сортов L состоит из бинарного отношения axiom ($\text{axiom}(\ell, \ell')$ постулирует, что терм ℓ имеет тип ℓ') и трёхместного отношения rule ($\text{rule}(\ell_S, \ell_T, \ell)$ постулирует, что если S — тип сорта ℓ_S , а T — тип сорта ℓ_T , то $\Pi(x : S).T$ — тип сорта ℓ).*

Класс 3 (Функциональная сигнатура сортов). *Сигнатура сортов функциональна, если: отношение axiom функционально (если $\text{axiom}(\ell, \ell_1)$ и $\text{axiom}(\ell, \ell_2)$, то $\ell_1 = \ell_2$); и отношение rule функционально по первой паре аргументов (если $\text{rule}(\ell_t, \ell_u, \ell_1)$ и $\text{rule}(\ell_t, \ell_u, \ell_2)$, то $\ell_1 = \ell_2$).*

Класс 4 (Полная сигнатура сортов). *Сигнатура сортов полна, если: каждый сорт находится в каком-то другом (для каждого ℓ существует ℓ' такой, что $\text{axiom}(\ell, \ell')$); и каждая пара сортов определяет сорт соответствующих типов функций (для каждого ℓ_t, ℓ_u существует ℓ такой, что $\text{rule}(\ell_t, \ell_u, \ell)$).*

1.4.3 Правила типизации

Здесь и далее фиксируется некоторая произвольная сигнатура сортов.

Определение 28 (Суждение типизации).

$$\begin{array}{c}
 \frac{}{\Gamma \vdash x : \Gamma !! x} \quad \frac{}{\Gamma \vdash s : s'} (s, s') \in \text{Ax} \quad \frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash U : s_U}{\Gamma \vdash \Pi(x : T).U : s} (s_T, s_U, s) \in \text{Rule} \\
 \\
 \frac{\Gamma \vdash T : s_T \quad \Gamma, x : T \vdash u : U}{\Gamma \vdash \lambda(x : T).u : \Pi(x : T).U} \quad \frac{\Gamma \vdash u : \Pi(x : T).U \quad \Gamma \vdash t : T}{\Gamma \vdash u \$ t : U[x := t]} \\
 \\
 \frac{\Gamma \vdash t : T \quad \Gamma \vdash U : s \quad T =_\beta U}{\Gamma \vdash t : U}
 \end{array}$$

Благодаря введённым ранее определениям для суждения типизации можно сформулировать и доказать усиленные версии следующих структурных свойств, из которых следуют их обычные версии, необходимые для доказательства сохранения типизации.

Лемма 53 (Сильное ослабление). *Если $\Gamma \vdash t : T$ и мы вставляем новый тип U в контекст, то переименованный терм и тип остаются корректно типированными:*
 $\text{insert}(\Gamma, U) \vdash \text{rename}(\text{wr})(t) : \text{rename}(\text{wr})(T)$.

Лемма 54 (Ослабление). *Если $\Gamma \vdash t : T$, то для любого типа U верно*
 $\Gamma, x : U \vdash \text{shift}(t) : \text{shift}(T)$.

Лемма 55 (Сильная подстановка). *Если $\Gamma \vdash t : T$ и $\text{shrink}(\Gamma) \vdash u : \text{squeeze}(\Gamma)$, то индексированная подстановка сохраняет суждение типирования, т.е. $\Gamma \triangleleft^i u \vdash t \triangleleft^i u : T \triangleleft^i u$.*

Лемма 56 (Подстановка). *Если $\Gamma, x : U \vdash t : T$ и $\Gamma \vdash u : U$, то $\Gamma \vdash t[x := u] : T[x := u]$.*

1.4.4 Леммы об инверсии

Доказательство сохранения типизации существенно опирается на существование лемм об инверсии, выводящих типизацию подтермов из типизации терма.

Лемма 57 (Инверсия сорта). *Если $\Gamma \vdash \ell : T$, то существует сорт ℓ' такой, что $\text{axiom}(\ell, \ell')$ и $T =_\beta \ell'$.*

Лемма 58 (Инверсия типа функций). *Если $\Gamma \vdash \Pi(x : T).U : V$, то существуют сорта ℓ_t, ℓ_u, ℓ такие, что*

$$\left\{ \begin{array}{l} \text{rule}(\ell_t, \ell_u, \ell) \\ \Gamma \vdash T : \ell_t \\ \Gamma, x : T \vdash U : \ell_u \\ V =_\beta \ell \end{array} \right.$$

Лемма 59 (Инверсия переменной). *Если $\Gamma \vdash x : T$, то $T =_\beta \Gamma !! x$.*

Лемма 60 (Инверсия лямбды). *Если $\Gamma \vdash \lambda(x : T).t : V$, то существуют сорт ℓ и терм U такие, что*

$$\left\{ \begin{array}{l} \Gamma \vdash T : \ell \\ \Gamma, x : T \vdash t : U \\ V =_\beta \Pi(x : T).U \end{array} \right.$$

Лемма 61 (Инверсия применения). *Если $\Gamma \vdash t \$ u : V$, то существуют термы U и T такие, что*

$$\left\{ \begin{array}{l} \Gamma \vdash t : \Pi(x : U).T \\ \Gamma \vdash u : U \\ V =_\beta T[x := u] \end{array} \right.$$

Теорема 4 (Однозначность типизации / Единственность типа). *Если сигнатура сортов функциональна, $\Gamma \vdash t : T$ и $\Gamma \vdash t : T'$, то $T =_\beta T'$. Иными словами, терм имеет не более одного типа с точностью до вычислительного равенства.*

1.4.5 Сохранение типизации

Мы подобрались к одному из самых важных свойств системы — редукция типизованного терма типизируется так же. Как и с конfluence'ностью, доказательство этого факта требует некоторых дополнительных построений; доказательство адаптировано из описания доказательства непротиворечивости Rosq на Rosq (TODO: ссылка).

Важным элементом доказательства сохранения типизации системы является определение одношаговой редукции на контекстах.

Определение 29 (β -редукция на контекстах).

$$\frac{T \xrightarrow{\beta} U}{\Gamma, x : T \xrightarrow{b} \Gamma, x : U} \qquad \frac{\Gamma \xrightarrow{b} \Delta}{\Gamma, x : T \xrightarrow{b} \Delta, x : T}$$

Заметим, что в наших правилах типизации никак не проверяется контекст; пришло время это сделать.

Определение 30 (Корректные контексты). Контекст Γ корректен (обозначение $\text{wf}(\Gamma)$) в одном из двух случаев:

1. Γ — пустой;
2. $\Gamma = \Gamma_0, x : T$; при этом Γ корректен и $\Gamma \vdash T : \ell$ для некоторого сорта ℓ .

Лемма 62 (Сильная редукция контекста). Если $\Gamma \vdash t : T$ и Γ корректен, а также $\Gamma \xrightarrow{b} \Delta$, то $\Delta \vdash t : T$. Иными словами, редукция типа в контексте сохраняет типизацию терма.

Лемма 63 (Редукция контекста). Если $\Gamma, x : U \vdash t : T$ и Γ корректен, а также $\Gamma \vdash U : \ell$ и $U \xrightarrow{\beta} U'$, то $\Gamma, x : U' \vdash t : T$.

Лемма 64 (Корректность типа). Если сигнатура сортов полна, $\Gamma \vdash t : T$ и Γ корректен, то существует сорт ℓ такой, что $\Gamma \vdash T : \ell$. Иными словами, тип в суждении типизации тоже типизируется некоторым сортом.

Теорема 5 (Сохранение типизации). Если сигнатура сортов полна, $\Gamma \vdash t : T$ и Γ корректен, а также $t \xrightarrow{\beta} t'$, то $\Gamma \vdash t' : T$.

1.5 О здравости чистых систем типов

1.5.1 Сильная нормализация

Так, мы выяснили, что наша система не совсем безнадёжна: вычисления не “расходятся”, и типизация сохраняется при вычислениях. Как видно из правил типизации, для практической реализации этой системы в виде средства интерактивных доказательств крайне желательно уметь проверять вычислительное равенство типов. Самый естественный способ — по теореме о характеристизации: упрощаем термы по β -редукции до тех пор, пока это возможно, и сравним результаты синтаксически. Но вдруг найдётся такой терм, который можно редуцировать бесконечно? Давайте исследуем свойство *нормализации*: терм нормализуется, если любая цепочка его упрощений конечна. Формально это определяется так:

Определение 31 (Доступность). Для отношения R на множестве A элемент x доступен (относительно R), если каждый y такой, что $x R y$, сам доступен.

Определение 32 (Сильная нормализуемость). Терм t сильно нормализуем (обозначение $SN(t)$), если он доступен относительно отношения одношаговой β -редукции.

Какие свойства выполнены для сильно нормализующихся термов?

Лемма 65 (Сильная нормализуемость по голове). Если $t \ \$$ u сильно нормализуем, то t сильно нормализуем.

Лемма 66 (Переименование сохраняет сильную нормализуемость). Если терм сильно нормализуем, то любое его переименование также сильно нормализуемо.

Определение 33 (Аджаст). Функция, которая для заданного m отображает индекс в $\text{Fin}(n)$ в индекс в $\text{Fin}(m + n)$ путём сдвига всех индексов на m .

Лемма 67 (Аджаст и сильная нормализуемость). Если терм, переименованный функцией adjust , сильно нормализуем, то исходный терм сильно нормализуем.

Определение 34 (Атомарные термы). Терм атомарен, если он есть: сорт; тип функций; переменная; или применение атомарной функции к атомарному аргументу.

Лемма 68 (Атомарность сохраняется при переименовании). Если терм атомарен, то любое его переименование также атомарно.

Лемма 69 (Атомарность сохраняется при β -преобразовании). Если $t \xrightarrow{\beta} t'$ и t атомарен, то t' также атомарен.

Лемма 70 (Сильная нормализуемость атомарного применения). Если t атомарен и сильно нормализуем, и u сильно нормализуем, то $t \ \$$ u сильно нормализуем.

Определение 35 (Слабое головное расширение). Отношение WHE (слабое головное расширение) на термах определяется так: если t слабо головно расширяется в t' , то $t \ \$$ u слабо головно расширяется в $t' \ \$$ u ; и если U и u сильно нормализуемы, то $\lambda U \ t \ \$$ u слабо головно расширяется в $t \ \triangleleft u$.

Другими словами, t слабо головно расширяется в t' , если t — это цепочка применений аргументов к сильно нормализуемому лямбда-выражению, где самый первый первый аргумент также сильно нормализуем, а t' — терм, получающийся после бета-редукции в голове этой цепочки.

Лемма 71 (Переименование коммутрует со слабым головным расширением). Если t слабо головно расширяется в t' , то любое переименование t слабо головно расширяется в переименование t' .

Лемма 72 (Слабое головное расширение и сильная нормализуемость). Если t слабо головно расширяется в t' и $t' \ \$$ u сильно нормализуем, то $t \ \$$ u сильно нормализуем.

В общем случае, проверка терма на нормализацию является неразрешимой задачей, поскольку в текущем представлении чистой системы типов очевидным образом выражается Тьюринг-полное нетипизированное лямбда-исчисление. Можно было бы надеяться, что типизация исправит ситуацию, но она всё ещё автоматически не гарантирует нормализацию, и какого-то классифицирующего критерия для ЧСТ, позволяющего выяснить (не)завершимость термов по сигнатуре сортов, нет. Тем не менее, для следующих классических чистых систем типов была доказана сильная нормализация:

- Исчисление конструкций:

$$\begin{aligned} S &= \{*, \square\} & Ax &= \{(*, \square)\} \\ \text{Rule} &= \{(*, *, *), (*, \square, \square), (\square, *, *), (\square, \square, \square)\} \end{aligned}$$

- Первая счётная иерархия *вселенных*:

$$\begin{aligned} S &= \{\mathcal{U}_\ell \mid \ell \in \mathbb{N}\} & \text{Ax} &= \{(\mathcal{U}_\ell, \mathcal{U}_{\ell+1}) \mid \ell \in \mathbb{N}\} \\ \text{Rule} &= \{(\mathcal{U}_\ell, \mathcal{U}_{\ell'}, \mathcal{U}_{\max(\ell, \ell')}) \mid \ell, \ell' \in \mathbb{N}\} \end{aligned}$$

- Счётная иерархия с импредикативным Prop:

$$\begin{aligned} S &= \{\text{Prop}\} \sqcup \{\mathcal{U}_\ell \mid \ell \in \mathbb{N}\} \\ \text{Ax} &= \{(\text{Prop}, \mathcal{U}_0)\} \sqcup \{(\mathcal{U}_\ell, \mathcal{U}_{\ell+1}) \mid \ell \in \mathbb{N}\} \\ \text{Rule} &= \{(\mathcal{U}_\ell, \text{Prop}, \text{Prop}) \mid \ell \in \mathbb{N}\} \sqcup \\ &\sqcup \{(\text{Prop}, s, s) \mid s \in S\} \sqcup \\ &\sqcup \{(\mathcal{U}_\ell, \mathcal{U}_{\ell'}, \mathcal{U}_{\max(\ell, \ell')}) \mid \ell, \ell' \in \mathbb{N}\} \end{aligned}$$

Кроме этого, о завершимости широкого класса чистых систем типов есть следующий отрицательный результат.

Теорема 6 (Girard'72). *Следующая ЧСТ (известная как System U) содержит не нормализующийся терм:*

$$\begin{aligned} S &= \{*, \square, \triangle\} & \text{Ax} &= \{(*, \square), (\square, \triangle)\} \\ \text{Rule} &= \{(*, *, *), (\square, *, *), (\triangle, *, *), (\square, \square, \square), (\triangle, \square, \square)\} \end{aligned}$$

Более того, этот терм имеет тип $\Pi(T : *).T$.

Из не-завершимости термов в System U следует не-завершимость любой другой системы, в которой System U выражается. Другими словами, «нормализующаяся ЧСТ не может поддерживать одновременно **и** импредикативный Prop, **и** полиморфные функции». Например, из парадокса Жирара выводится противоречивость ещё более простой ЧСТ, известной как Type-in-Type:

$$S = \{*\} \quad \text{Ax} = \{(*, *)\} \quad \text{Rule} = \{(*, *, *)\}$$

1.5.2 Теоретико-множественная семантика

Другое свойство, которое стоит требовать от системы, претендующей на фундамент системы доказательств — каноничность: всякий терм вычисляется до некоторого своего “значения”. Кроме сильной нормализации, можно также исследовать и другое свойство, наиболее важное для формализации математики — наличие теоретико-множественной модели. У счётных иерархий она есть; правда, такая семантика интерпретирует $\mathcal{U}_\ell$ как ℓ -й недостижимый кардинал.

1.6 О здравости чистой системы типов, основанной на рангах

(TODO: завершить формализацию)

Определение 36. Рангом функции называется максимальная вложенность Π в позиции аргумента:

$$\text{rk}(\Pi(x : T).U) = \max(\text{rk } T + 1, \text{rk } U)$$

Рассмотрим $S = \{\text{rk}_n \mid n \in \mathbb{N}\}$, и

$$\text{Rule} = \{(\text{rk}_m, \text{rk}_n, \text{rk}_{\max(m+1, n)}) \mid m, n \in \mathbb{N}\}.$$

В качестве Ax можно взять цепочку $\text{rk}_0 : \text{rk}_1 : \text{rk}_2 : \dots$, но получится система, строго менее удобная, чем обычная иерархия вселенных. Попробуем поступить следующим образом:

$$Ax = \{(\text{rk}_n, \text{rk}_0) \mid n \in \mathbb{N}\}$$

В частности, $\text{rk}_0 : \text{rk}_0$. Казалось бы, такая система не должна нормализовываться из-за парадокса Жирара; однако он здесь не применим потому, что для его кодирования в системе необходим как минимум один сорт, замкнутый относительно типов функций; такого сорта в нашей системе нет.

1.6.1 Сильная нормализация

Мы доказываем сильную нормализацию с помощью стандартной техники *кандидатов редуцируемости*, используя тот факт, что система **предикативна**: правило произведения строго повышает уровень сорта, поэтому, хотя $\text{rk}_0 : \text{rk}_0$, сорт rk_0 не замкнут относительно произведений.

Определение 37 (Кандидаты редуцируемости для сортов). *Определим кандидаты $C_n \subseteq \text{SN}$ для каждого сорта rk_n индукцией по n .*

- $n = 0$: $C_0 := \{t \in \text{SN} \mid \cdot \vdash t : \text{rk}_0\}$.
- $n > 0$: $C_n := \{t \in \text{SN} \mid \cdot \vdash t : \text{rk}_n \wedge \text{Cond}_n(t)\}$.

Условие $\text{Cond}_n(t)$ гласит следующее: если t редуцируется в $\Pi(x : A).B$, то

$$A \in C_m, \quad \forall u \in \llbracket \text{nf}(A) \rrbracket, \quad B[x := u] \in C_k, \quad \max(m + 1, k) = n.$$

Здесь $\llbracket \text{nf}(A) \rrbracket$ — кандидат, являющийся интерпретацией замкнутой нормальной формы элемента C_m , определённый ниже.

Это определение корректно, поскольку в функции ранга n область определения имеет ранг $m < n$, так что C_m уже определено. Множество значений может иметь ранг $k \leq n$, но оно встречается как подтерм, поэтому рекурсия структурна по термам.

Определение 38 (Кандидаты редуцируемости канонических типов). *Для завершения определения кандидатов C_n нам достаточно определить интерпретацию замкнутых нормальных форм элементов C_m . Таких всего два вида:*

$$\begin{aligned} \llbracket \text{rk}_n \rrbracket &:= C_n, \\ \llbracket \Pi x:A.B \rrbracket &:= \{t \in \text{SN} \mid \forall u \in \llbracket A \rrbracket, \quad t \, u \in \llbracket \text{nf}(B[x := u]) \rrbracket\}. \end{aligned}$$

Перед формулировкой Основной леммы нам понадобится ещё одно определение: корректности подстановки и контекста.

Определение 39. *Контекст Γ и подстановка $\sigma : \text{Fin}(|\Gamma|) \rightarrow \text{Term}(0)$ являются корректными (обозначается $\sigma :: \Gamma$), если:*

1. $w : \text{wf}(\Gamma)$;
2. Для любого x верно, что $\sigma(x) : (\Gamma !! x)[\sigma]$;

3. Если в $w : \mathbf{wf}(\Gamma)$ указано, что $T : \mathbf{rk}_k$, то $T[\sigma] \in C_k$;
4. Для любого x верно, что $\sigma(x) \in \llbracket \mathbf{nf}((\Gamma !! x)[\sigma]) \rrbracket$.

Теперь мы готовы сформулировать и доказать Основную лемму.

Лемма 73 (Основная лемма).

$$\left\{ \begin{array}{l} \Gamma \vdash t : T \\ \sigma :: \Gamma \end{array} \right\} \Rightarrow \left\{ \begin{array}{l} \exists n. T[\sigma] \in C_n \\ t[\sigma] \in \llbracket \mathbf{nf}(T[\sigma]) \rrbracket \end{array} \right\}$$

Доказательство. Индукцией по выводу $\Gamma \vdash t : A$.

- **Переменная:** непосредственно из $\sigma :: \Gamma$.
- **Сорт:** $\mathbf{rk}_n : \mathbf{rk}_0$. Нужно $\mathbf{rk}_n, \mathbf{rk}_0 \in C_0$, что выполнено.
- **Тип функций:** Если $\Gamma \vdash A : \mathbf{rk}_m$ и $\Gamma, x : A \vdash B : \mathbf{rk}_n$, то $\Pi x:A.B : \mathbf{rk}_{\max(m+1,n)}$. По индукции $A[\sigma] \in C_m$ и для всех $u \in \llbracket A[\sigma] \rrbracket$ имеем $B[\sigma, x := u[\sigma]] \in C_n$. Следовательно, $(\Pi x:A.B)[\sigma] \in C_{\max(m+1,n)}$; кроме этого нужно $\mathbf{rk}_{\max(m+1,n)} \in C_0$, что выполнено.
- **Применение:** Если $\Gamma \vdash f : \Pi x:A.B$ и $\Gamma \vdash u : A$, то $f u : B[x := u]$. По индукции $(\Pi(x:A).B)[\sigma] \in C_n$, $f[\sigma] \in \llbracket (\Pi x:A.B)[\sigma] \rrbracket$ и $u[\sigma] \in \llbracket A[\sigma] \rrbracket$. По определению C_n получаем $B[\sigma, x := u] \in C_m$, а по определению интерпретации Π получаем $(f u)[\sigma] \in \llbracket B[\sigma, x := u] \rrbracket$.
- **Абстракция:** Если $\Gamma \vdash A : \mathbf{rk}_n$ и $\Gamma, x : A \vdash t : B$, то $\Gamma \vdash \lambda x:A.t : \Pi x:A.B$. $(\Pi(x:A).B)[\sigma] \in C_m$ верно по индукции; чтобы показать $(\lambda x:A.t)[\sigma] \in \llbracket (\Pi x:A.B)[\sigma] \rrbracket$, возьмём произвольное $u \in \llbracket A[\sigma] \rrbracket$. Тогда

$$(\lambda x:A.t)[\sigma] u \xrightarrow{\beta} t[\sigma, x := u].$$

По индукции $t[\sigma, x := u] \in \llbracket B[u] \rrbracket$, а кандидаты замкнуты относительно редукции, поэтому $(\lambda x:A.t) u \in \llbracket B[u] \rrbracket$. Кроме того, $\lambda(x:A).t$ сильно нормализуем, так как является головой сильно нормализуемого терма.

- **Конверсия:** Если $\Gamma \vdash t : A$ и $\Gamma \vdash B : \mathbf{rk}_n$, а также $A =_{\beta} B$, то $\Gamma \vdash t : B$. Тогда $B[\sigma] \in C_n$ верно по предположению индукции; кроме этого, по предположению индукции $t[\sigma] \in \llbracket A[\sigma] \rrbracket = \llbracket B[\sigma] \rrbracket$.

Таким образом, всякий хорошо типизированный терм лежит в некотором кандидате, а значит, и в SN. $\square$

1.6.2 Теоретико-множественная семантика

У нашей системы на основе рангов очень интересная теоретико-множественная семантика. В силу того, что $\mathbf{rk}_0 : \mathbf{rk}_0$, мы не можем интерпретировать суждение о типизации $t : T$ как $\llbracket t \rrbracket \in \llbracket T \rrbracket$; вместо этого мы построим два разных отображения, $\llbracket _ \rrbracket$ и $\llbracket _ \rrbracket$, и покажем, что если $t : T$, то $\llbracket t \rrbracket \in \llbracket T \rrbracket$ (идея такого построения вдохновлена подходом из статьи A Generic Normalization Proof for Pure Type Systems, TODO: link).

TODO: завершить описание.

В целом, обе интерпретации термов совпадают; единственное существенное отличие в том, как они интерпретируют $\mathbf{rk}_0$:

$$\begin{aligned} \llbracket \mathbf{rk}_0 \rrbracket &= \mathbb{N}; \\ \llbracket \mathbf{rk}_0 \rrbracket &= \{ \llbracket \mathbf{rk}_n \rrbracket \mid n \in \mathbb{N} \}. \end{aligned}$$

Хотим обратить внимание, что пока непонятно, интерпретируется ли в нашей системе сколько-нибудь содержательная теория множеств, так что ничего сильно удивительного в этом результате нет: это лишь небольшое обобщение результата о сильной нормализуемости системы.

2. К системам типов, способных к самореинтерпретации

Мы убеждены, что системы типов должны уметь переинтерпретировать собственные термины в произвольной внутренней модели: вооружившись единственным инструментом — реинтерпретацией — можно выразить широкий круг возможностей, таких как полиморфизм вселенных, внешняя параметричность, возможность вести синтетическую математику непосредственно внутри системы и многое другое. В этой связи мы исследуем пределы самореинтерпретации, представляем две системы типов, способные к ней (НМ-R и MLTT-R), доказываем их базовые свойства и приводим несколько примеров полезных внутренних моделей.

2.1 Введение

Рассмотрим следующую хорошо известную функцию, применяющую заданное преобразование к элементам списка, написанную на Agda:

```
map : ∀ {A B : Set} → (A → B) → List A → List B
map f [] = []
map f (x :: xs) = f x :: map f xs
```

Наслаждаясь и композиционностью этой функции, и простотой её типа, мы хотели бы максимально переиспользовать её определение. В некоторой степени это было бы, безусловно, возможно: ведь `map` полиморфна относительно исходного и целевого типов, так что мы можем подставлять туда любые желаемые типы. Однако, если бы нам попалась волшебная лампа с джином теории типов, мы попросили бы его исполнить три желания:

1. Переиспользовать `map` на более высоких уровнях вселенных: например, имея *оператор типов*, применить `map` к *списку типов*:

$$(\text{Set} \rightarrow \text{Set}) \rightarrow \text{List Set} \rightarrow \text{List Set}$$

2. Автоматически получить доказательство свободной теоремы для `map`, то есть некоторый терм типа

$$\begin{aligned} &\forall A A' (Q : A \rightarrow A' \rightarrow \text{Prop}), \forall B B' (R : B \rightarrow B' \rightarrow \text{Prop}), \\ &\quad \forall f f', (\forall x x', Q x x' \rightarrow R(fx) (f'x')) \rightarrow \\ &\quad \forall (xs : \text{List } A) (xs' : \text{List } A'), Q^* xs xs' \rightarrow R^*(\text{map } f xs) (\text{map } f' xs'); \end{aligned}$$

3. По данным группоидам A и B и функтору $f : A \rightarrow B$ построить функтор $\text{map } f$ между группоидами $\text{List } A$ и $\text{List } B$.

Первое желание, разумеется, исполняется: достаточно доработать определение, сделав его полиморфным относительно уровней. Хотя теорема из второго желания уже верна в системах с параметричностью, получить её доказательство *внутри* системы без участия человека можно лишь в тех системах типов, где параметричность интернализирована [7, 5]. Третье желание пока приходится исполнять вручную (или с привлечением метапрограммирования [27]), хотя для этого требуется лишь пара индукций по списку: несколько для определения экземпляра группоида для List и ещё несколько для доказательства функториальности map .

Мы предлагаем иной подход. Прежде чем дорабатывать определение map , вспомним всю метатеоретическую мощь, которой наша система типов уже обладает. Первой на ум приходит упомянутая выше параметричность полиморфных определений; обратим внимание, что она получается путём задания *реляционной интерпретации* суждений выводимости [30]. Действительно, некоторые из наиболее интересных метатеоретических результатов выражаются через построение определённой *модели* системы типов: синтетические результаты в теории гомотопий опираются на $(\infty, 1)$ -группоидную модель (гомотопической) теории типов [19, 4]; применение Макбрайдом операции смещения для полиморфизма вселенных [24, 25] можно рассматривать как построение модели системы типов в более высоких вселенных её же самой, а затем интерпретацию смещённых конструкций в ней. Тогда все перечисленные выше желания могли бы быть исполнены путём построения подходящей модели системы типов *внутри* неё же (отсюда наш термин *внутренние модели*) и последующего инстанцирования map в ней. Ниже мы будем называть системы типов с возможностью самореинтерпретации *рефлексивными*.

2.1.1 Вклад

- Наша главная цель — построить реализуемую рефлексивную систему типов с зависимыми типами. Основная трудность при этом возникает из-за результата о невозможности, обсуждаемого в разделе 2.2, который вынуждает нас ограничить мощность реинтерпретации, чтобы сохранить алгоритмичность проверки типов.
- Ради удобства изложения мы сначала предлагаем рефлексивную систему типов на основе системы типов Хиндли — Милнера, названную НМ-R (раздел 2.3). Мы доказываем, что НМ-R корректна, консервативна относительно просто типизированного лямбда-исчисления, и что глобальный вывод типов алгоритмичен (раздел 2.3.1).
- Опираясь на наработки, полученные при разработке НМ-R, мы наконец готовы погрузиться в рефлексивную версию теории типов Мартина-Лёфа, именуемую MLTT-R (раздел 2.4). Мы также доказываем корректность и консервативность MLTT-R (раздел 2.4.2).
- Как и планировалось, благодаря реинтерпретации MLTT-R объединяет и ограниченную форму полиморфизма вселенных, и *внутреннюю* *внешнюю* параметричность. И, разумеется, MLTT-R можно использовать как систему для синтетической математики с внутренним переиспользованием синтетических результатов (раздел 2.4.3).

2.2 Теорема о невозможности внутренней интерпретации

Итак, мы хотим построить рефлексивную систему типов, то есть систему, в которой пользователь может как определить модель системы внутри неё, так и применить её к

корректно типизированным термам, получив их внутренние интерпретации. Разумеется, нам хотелось бы, чтобы определяемые пользователем модели сохраняли дефиниционные равенства системы; в системе с зависимыми типами это естественно достигается с помощью типа тождества, позволяющего явно указать соответствующие равенства внутри модели. Было бы также хорошо, если бы перед реинтерпретацией терма система сначала приводила его к нормальной форме, чтобы избавиться от артефактов построения в результате интерпретации. Наконец, в системе с зависимыми типами естественно ожидать редукцию открытых термов, так что мы хотим, чтобы наша интерпретация вычисляла значения на открытых термах.

Доказываемая ниже теорема утверждает, что в одной системе невозможно иметь все три свойства одновременно.

Сначала пара определений. Мы будем работать в рамках классической типотейоретической схемы, которая вводит контексты Γ , термы t, t' и типы T , связанные отношением типизации $\Gamma \vdash t : T$; вычисление задаётся одношаговым отношением редукции $t \rightsquigarrow t'$ на термах.

Определение 40 (Наивная вычислительная реинтерпретация). *Оператор типов $F[_]$ называется наивной вычислительной реинтерпретацией типов, если он сохраняет вычисления на типах (то есть, если на типах существует отношение редукции ($\rightsquigarrow$) и $T \rightsquigarrow T'$, то $F[T] \rightsquigarrow^* F[T']$) и вычисляется как примитивно рекурсивная функция от нормальных форм типов. Аналогично, терм с дыркой $\llbracket _ \rrbracket$ называется наивной вычислительной реинтерпретацией термов, если он сохраняет вычисления на термах, вычисляется как примитивно рекурсивная функция от нормальных форм термов и, если $\Gamma \vdash t : T$, то $\Gamma \vdash \llbracket t \rrbracket : F[T]$.*

Теорема 7 (Теорема о невозможности наивной реинтерпретации). *В конфлюэнтной системе типов с типом функций и наивной вычислительной реинтерпретацией, если редукция коммутрует с подстановкой, то для произвольных корректно типизированных s и t существует и такой, что:*

$$\text{app}[\lambda x.t][s] \rightsquigarrow^* u \leftarrow^* \llbracket t[x := s] \rrbracket$$

где app — действие $\llbracket _ \rrbracket$ на аппликации.

Следовательно, $\text{app}[\lambda x.t][s]$ и $\llbracket t[x := s] \rrbracket$ дефиниционно эквивалентны, что требует бета-редукции в модели выполняться дефиниционно, а не пропозиционально, что прямо противоречит нашему желаемому результату. Аналогичные теоремы можно доказать для любой другой редукции, присутствующей в системе; иными словами, **пропозициональное равенство недостаточно сильно**. В качестве доказательства приведём конкретный пример:

Доказательство. Даны корректно типизированные $\Gamma \vdash s : S$ и $\Gamma, x : S \vdash t : T$. Рассмотрим следующий терм:

$$\Gamma, x : S \rightarrow T, y : S \vdash \llbracket xy \rrbracket : F[T]$$

Терм под интерпретацией находится в нормальной форме, следовательно, весь терм вычисляется в $\text{app}[\llbracket x \rrbracket][\llbracket y \rrbracket]$. Подставляя $\lambda x.t$ вместо x и s вместо y , получаем $\text{app}[\lambda x.t][s]$.

С другой стороны, если сначала выполнить подстановку, мы получили бы $\Gamma \vdash \llbracket (\lambda x.t)s \rrbracket : F[T]$. Поскольку наивная реинтерпретация сохраняет редукцию, это редуцируется в $\llbracket t[x := s] \rrbracket$.

Поскольку редукция коммутует с подстановкой, а система конфлюэнтна, должно существовать такое u , в которое редуцируются оба терма. $\square$

Доказательство выше эксплуатирует открытость исходного терма; если мы ограничим реинтерпретацию вычислением только на замкнутых термах, мы сможем получить корректную систему без требования дефиниционных равенств в моделях, как это демонстрирует наша система без зависимых типов НМ-R, описанная в разделе 2.3. Однако для системы с зависимыми типами наша теорема о невозможности оставляет нам три варианта:

1. Ограничить модели теми, в которых равенства выполняются дефинициально, а не пропозиционально.
2. Не выполнять реинтерпретацию, пока терм не является замкнутым. Для рассуждений над открытыми терминами уравнения, описывающие поведение интерпретации, выставляются как пропозициональные аксиомы.
3. Не редуцировать открытые термы под реинтерпретацией вовсе, выполняя интерпретацию как есть. Для упрощения результата реинтерпретации можно переписывать его с помощью равенств, присутствующих в модели.

Если рассматривать существующие работы о дисплейсментах [24, 25], внутренней параметричности [5] и отображённой теории типов [22] через призму реинтерпретации внутри фиксированной модели (или семейства моделей), то видно, что соответствующие модели действительно выполняют равенства дефинициально; следовательно, по-видимому, необходимо принять первый подход. К тому же он наиболее удовлетворителен с вычислительной точки зрения. Однако, если Вам нужна корректная система, в которой модели могут определяться пользователем, необходимо ограничивать модели через дисциплину типизации. Используя известные на данный момент методики [8], такое ограничение моделей потребовало бы определения типа моделей как типа алгебр индуктивно-индуктивно-рекурсивного типа (или даже индуктивно-индуктивно-рекурсивного симплициального типа, для унивалентных теорий типов), что значительно выходит за рамки данной статьи.

Из двух оставшихся вариантов последний лучше подходит для практических систем с зависимыми типами, поскольку он надлежащим образом вычисляет значения на открытых термах. К тому же он не ограничивает класс возможных моделей, при этом предоставляя преимущества первого подхода: если модель такова, что уравнения выполняются дефинициально, то переписывание результата реинтерпретации для его редукции не требуется. Единственный недостаток состоит в том, что, возможно, редукции *после* интерпретации обходятся дороже, чем те, которые мы могли бы выполнить *до* неё.

2.3 Почти просто типизированный почти рефлексивный язык: НМ-R

Мы начинаем с более простого языка, чтобы проработать все детали и разобраться, как правильно реинтерпретировать термы, прежде чем переходить к системе с зависимыми типами. Простейший язык, для которого нам известно множество интересных моделей, — это просто типизированное лямбда-исчисление с произведениями, которое мы в течение всей статьи будем называть “STLC”; в этом разделе мы представим язык, способный интерпретировать термы STLC как конструкции в моделях STLC, которые могут быть определены в нашем языке.

Итак, что же представляют собой модели STLC? Помимо обычной теоретико-множественной семантики, где $S \rightarrow T$ интерпретируется как функция из S в T , а $S \times T$ — как декартово произведение, существует теоретико-доказательственная семантика, которая связывает термы типа T с интуиционистскими доказательствами пропозиции $\llbracket T \rrbracket$, где $(\rightarrow)$ теперь понимается как импликация, а $(\times)$ — как конъюнкция. Это, конечно, классическое соответствие Карри–Говарда [21]; обе семантики — теоретико-множественная и доказательственная — обобщаются полным описанием моделей STLC, данным Йоханом Ламбеком [23]. В соответствии с этим *соответствием Карри–Говарда–Ламбека*, модели STLC суть именно декартово замкнутые категории. Им можно дать множество различных определений; мы будем использовать определение 41.

Определение 41 (Декартово замкнутая категория). *Декартово замкнутая категория — это следующие данные:*

1. Множество объектов Ob и семейство морфизмов $\text{Hom}(S, T)$, индексированное $S, T \in \text{Ob}$, где $f \in \text{Hom}(S, T)$ обозначается как $f : S \rightarrow T$. Кроме того, S называется исходным объектом f , а T — целевым объектом f .
2. Для каждого $A \in \text{Ob}$ существует тождественный морфизм $\text{id}_A : A \rightarrow A$; для каждого $f : A \rightarrow B$ и $g : B \rightarrow C$ существует их композиция $g \circ f : A \rightarrow C$. Если объект-цель тождественного морфизма ясен из контекста, нижний индекс опускается. Кроме того, для произвольных морфизмов f, g, h с согласованными исходными и целевыми объектами выполняются следующие свойства:

$$\text{id} \circ f = f = f \circ \text{id} \qquad (f \circ g) \circ h = f \circ (g \circ h)$$

3. Существует терминальный объект $\top \in \text{Ob}$ такой, что для каждого $A \in \text{Ob}$ существует **единственный** морфизм $!_A : A \rightarrow \top$.
4. Для каждого $A, B \in \text{Ob}$ существует их произведение $A \times B$ вместе с проекциями $\pi_1 : A \times B \rightarrow A$ и $\pi_2 : A \times B \rightarrow B$; для каждого объекта $C \in \text{Ob}$ и пары морфизмов $f : C \rightarrow A$ и $g : C \rightarrow B$ существует **единственный** морфизм пары $[f; g] : C \rightarrow A \times B$ такой, что выполняются следующие свойства:

$$\pi_1 \circ [f; g] = f \qquad \pi_2 \circ [f; g] = g$$

5. Для каждого $A, B \in \text{Ob}$ существует их экспоненциальный объект B^A вместе с морфизмом применения $\text{ev} : B^A \times A \rightarrow B$; для каждого объекта $C \in \text{Ob}$ и морфизма $f : C \times A \rightarrow B$ существует **единственный** морфизм абстракции $\lambda f : C \rightarrow B^A$ такой, что для произвольного $g : C \rightarrow A$ выполняется следующее свойство:

$$\text{ev} \circ [\lambda f; g] = f \circ [\text{id}_C; g]$$

И тогда корректно типизированный терм $\Gamma \vdash t : T$ интерпретируется как морфизм $\llbracket t \rrbracket : \llbracket \Gamma \rrbracket \rightarrow \llbracket T \rrbracket$, где $\llbracket \Gamma \rrbracket$ — вложенное влево произведение интерпретаций типов из Γ :

$$\llbracket x : A, y : B, z : C \rrbracket = ((\top \times \llbracket A \rrbracket) \times \llbracket B \rrbracket) \times \llbracket C \rrbracket.$$

Тип единицы теперь — терминальный объект; произведение типов теперь — произведение объектов; тип стрелки — экспоненциальный объект. Уравнения в (2) необходимы для выполнения структурных свойств подстановки; уравнения в (4) соответствуют редукциям пар; уравнение в (5) выполняет β -редукцию. Свойства **уникальности** соответствуют η -эквивалентности.

Например, в нашем языке $\llbracket t \rrbracket \mathcal{M}$ — это оператор, реинтерпретирующий терм t во внутренней модели $\mathcal{M}$. Для следующего терма:

$$\cdot \vdash \lambda fxy.(fx, y) : (A \rightarrow B) \rightarrow A \rightarrow C \rightarrow B \times C$$

вычисление происходит следующим образом:

$$\begin{aligned} \llbracket \lambda fxy.(fx, y) \rrbracket \mathcal{M} &= \\ &= \text{lam}(\text{lam}(\text{lam}(\text{pair}(\text{eval} \circ \text{pair}(\text{proj}_2 \circ \text{proj}_1 \circ \text{proj}_1)(\text{proj}_2 \circ \text{proj}_1)) \text{proj}_2))) \end{aligned}$$

MonoType	S, T, C, P, E, TT	$::= \alpha$	переменная типа
		$ \top$	тип единицы
		$ S \times T$	произведение типов
		$ S \rightarrow T$	тип стрелки
PolyType	σ	$::= T$	моноформный тип
		$ \forall \alpha. \sigma$	универсальный тип
Term	s, t	$::= x$	переменная
		$ ()$	единичное значение
		$ (s, t)$	пара
		$ \pi_1 t$	первая проекция
		$ \pi_2 t$	вторая проекция
		$ \lambda x. t$	абстракция
		$ t s$	применение
		$ \text{let } x = s \text{ in } t$	полиморфное связывание
		$ x^{\mathcal{M}}$	реинтерпретация
Model	$\mathcal{M}$	$::= \langle T, t, \dots \rangle$	определение модели (см. правила типизации)

Рис. 2.1: Типы и термы НМ-R

где `lam`, `pair` и `eval` — морфизм абстракции, морфизм пары и морфизм применения, взятые из $\mathcal{M}$ и определённые таким же образом, как в определении 41, переведённые в типы нашей системы.

Теперь перейдём к самому проектированию нашего языка. Как обсуждалось в разделе 2.2, для независимого случая, с которым мы сейчас имеем дело, мы будем интерпретировать только нормальные формы замкнутых терминов. В частности, это означает, что нам фактически не нужно, чтобы уравнения и **уникальности** из определения 41 выполнялись в наших внутренних моделях! Такой подход не нарушает ни одного из метатеоретических свойств, рассмотренных в разделе 2.3.1. Кроме того, для простоты примера мы будем рассматривать только модели, в которых объекты — это типы. Тогда интернализация определения 41 потребует 1) операторов типов для определения типов морфизмов, произведений и экспоненциальных объектов; 2) полиморфных функций для операционализации $\forall \exists$ конструкций в модели. На самом деле, мы от операторов типов тоже избавимся, поскольку нам достаточно лишь типов в контексте, которые затем подставляются в определение модели для проверки операций на уровне терминов. Таким образом, наш язык представляет собой небольшое расширение классической системы типов Хиндли — Милнера, отсюда и название НМ-R; грамматические правила собраны на рис. 2.1. Обратите внимание, что, несмотря на наше описание, грамматика не определяет оператор реинтерпретации $\llbracket t \rrbracket_{\mathcal{M}}$ для произвольного термина t ; вместо этого реинтерпретации подлежат только переменные. В этой системе реинтерпретация — не что иное, как полная функция на замкнутых значениях.

Определение 42 (Значения в НМ-R). *Значения обозначаются метаварiableными u, v в BNF-грамматике ниже. Метаварiableная n обозначает нейтральные термы:*

$$\begin{aligned} u, v &::= n \mid () \mid (u, v) \mid \lambda x. v \\ n &::= x \mid \pi_i n \mid n v \end{aligned}$$

Таким образом, для определения реинтерпретации достаточно рассмотреть термы STLC:

$$\begin{array}{c}
\frac{s \rightsquigarrow s'}{(s, t) \rightsquigarrow (s', t)} \quad \frac{t \rightsquigarrow t'}{(s, t) \rightsquigarrow (s, t')} \quad \frac{t \rightsquigarrow t'}{\pi_i t \rightsquigarrow \pi_i t'} \quad \pi_i(t_1, t_2) \rightsquigarrow t_i \\
\\
\frac{t \rightsquigarrow t'}{\lambda x. t \rightsquigarrow \lambda x. t'} \quad \frac{s \rightsquigarrow s'}{s t \rightsquigarrow s' t} \quad \frac{t \rightsquigarrow t'}{s t \rightsquigarrow s t'} \quad (\lambda x. t) s \rightsquigarrow t[x := s] \\
\\
\frac{s \rightsquigarrow s'}{\text{let } x = s \text{ in } t \rightsquigarrow \text{let } x = s' \text{ in } t} \quad \frac{t \rightsquigarrow t'}{\text{let } x = s \text{ in } t \rightsquigarrow \text{let } x = s \text{ in } t'} \\
\\
\frac{\text{fv}(v) = \emptyset}{\text{let } x = v \text{ in } t \rightsquigarrow t[x := v]}
\end{array}$$

Рис. 2.2: Правила вычислений в НМ-R

Определение 43 (Реинтерпретация термов НМ-R).

$$\llbracket () \rrbracket^{\mathcal{M}} / \gamma = \text{cns}t \quad (2.1)$$

$$\llbracket (s, t) \rrbracket^{\mathcal{M}} / \gamma = \text{pair } \llbracket s \rrbracket^{\mathcal{M}} / \gamma \llbracket t \rrbracket^{\mathcal{M}} / \gamma \quad (2.2)$$

$$\llbracket \pi_i t \rrbracket^{\mathcal{M}} / \gamma = \text{comp proj}_i \llbracket t \rrbracket^{\mathcal{M}} / \gamma \quad (2.3)$$

$$\llbracket x \rrbracket^{\mathcal{M}} / \gamma = \gamma x \quad (2.4)$$

$$\llbracket \lambda x. t \rrbracket^{\mathcal{M}} / \gamma = \text{lam } \llbracket t \rrbracket^{\mathcal{M}} / [\vec{y} := \text{comp proj}_1(\gamma \vec{y}), x := \text{proj}_2] \quad (2.5)$$

$$\llbracket t s \rrbracket^{\mathcal{M}} / \gamma = \text{compeval } (\text{pair } \llbracket t \rrbracket^{\mathcal{M}} / \gamma \llbracket s \rrbracket^{\mathcal{M}} / \gamma) \quad (2.6)$$

где *cns*t, *pair* и другие операции модели взяты из $\mathcal{M}$, а γ — подстановка.

Вот наш подход к обеспечению принципа, согласно которому *интерпретируются только замкнутые термы*: как только терм s , связанный полиморфным let-связыванием $\text{let } x = s \text{ in } t$, становится замкнутым и приводится к нормальной форме, только тогда он подставляется в t , а всякий раз, когда он подставляется в $x^{\mathcal{M}}$, он немедленно переинтерпретируется. Эта операция обозначается как $t[x := s]$ в наших правилах вычисления, данных на рис. 2.2. Как и прежде, она ведёт себя как обычная подстановка во всех случаях, за исключением переменных, находящихся под переинтерпретацией:

$$(x^{\mathcal{M}}) [x := s] = s^{\mathcal{M}} / \varepsilon \quad (\varepsilon \text{ — пустая подстановка})$$

Легко заметить, что вычисление зависло бы, если бы let-связанные термы содержали переменные, пришедшие из неприменённых λ -абстракций. Наши правила типизации, представленные на рис. 2.3, защищают нас от этого, позволяя ссылаться только на let-связанные термы внутри других let-связанных термов. И вновь, помимо этого и правила типизации моделей, здесь нет ничего отличного от стандартных декларативных правил типизации системы типов Хиндли — Милнера [12].

Чтобы проверять модели, мы немного злоупотребляем нашей системой типов, чтобы не вводить операторов типов и не усложнять её: типы морфизмов C , объектов-произведений P и экспоненциалов E задаются как открытые мономорфные типы, которые могут использовать две типовые переменные, 0 и 1. Этот факт отражается в синтаксисе модели с помощью нотации $\Lambda 01.C$, $\Lambda 01.P$ и $\Lambda 01.E$ соответственно. 0 и 1 затем подставляются при типизации операций модели (нотация $T[S_1, S_2]$ является сокращением для $T[0 := S_1, 1 := S_2]$). Терминальный объект — это просто тип с именем TT .

$$\begin{array}{c}
\Gamma \vdash () : \top \qquad \frac{\Gamma \vdash s : S \quad \Gamma \vdash t : T}{\Gamma \vdash (s, t) : S \times T} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_1 t : S} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_2 t : T} \\
\\
\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \qquad \frac{\Gamma, x : S \vdash t : T}{\Gamma \vdash \lambda x. t : S \rightarrow T} \qquad \frac{\Gamma \vdash t : S \rightarrow T \quad \Gamma \vdash s : S}{\Gamma \vdash t s : T} \\
\\
\frac{\forall \Gamma \vdash s : S \quad \text{fv}(S) = \vec{\alpha} \quad \forall \Gamma, x : \forall \vec{\alpha}. S, \Delta \vdash t : T}{\forall \Gamma, \Delta \vdash \text{let } x = s \text{ in } t : T} \qquad \frac{x : \forall \vec{\alpha}. T \in \forall \Gamma \quad \mathcal{M} \text{ model}}{\forall \Gamma, \Delta \vdash x^{\mathcal{M}} : \llbracket T \rrbracket_{[\vec{\alpha} := \vec{s}]}^{\mathcal{M}}} \\
\\
\frac{\begin{array}{l} \text{fv}(C) \cup \text{fv}(P) \cup \text{fv}(E) \subseteq \{0, 1\} \quad \Gamma \vdash \text{id} : C[\alpha, \alpha] \quad \Gamma \vdash \text{comp} : C[\beta, \gamma] \rightarrow C[\alpha, \beta] \rightarrow C[\alpha, \gamma] \\ \text{fv}(TT) = \emptyset \quad \Gamma \vdash \text{cnst} : C[\alpha, TT] \quad \Gamma \vdash \text{pair} : C[\gamma, \alpha] \rightarrow C[\gamma, \beta] \rightarrow C[\gamma, P[\alpha, \beta]] \\ \Gamma \vdash \text{proj}_1 : C[P[\alpha, \beta], \alpha] \quad \Gamma \vdash \text{proj}_2 : C[P[\alpha, \beta], \beta] \\ \Gamma \vdash \text{lam} : C[P[\gamma, \beta], \alpha] \rightarrow C[\gamma, E[\alpha, \beta]] \quad \Gamma \vdash \text{eval} : C[P[E[\alpha, \beta], \beta], \alpha] \end{array}}{\Gamma \vdash \langle \Lambda 01.C, \text{id}, \text{comp}, TT, \text{cnst}, \Lambda 01.P, \text{pair}, \text{proj}_1, \text{proj}_2, \Lambda 01.E, \text{lam}, \text{eval} \rangle \text{ model}}
\end{array}$$

Рис. 2.3: Декларативные правила типизации НМ-Р

Чтобы проверять переинтерпретацию, нам нужно определить, как модель действует на мономорфные типы. Поскольку на них не выполняется вычисление, переинтерпретация типов снова определяется как рекурсивная функция от синтаксиса:

Определение 44 (Переинтерпретация типов в НМ-Р). Пусть $\mathcal{M}$ — корректная модель, а σ — подстановка мономорфных типов. Тогда интерпретация типа в модели есть следующая функция:

$$\llbracket T \rrbracket_{\sigma}^{\mathcal{M}} = C[TT, T_{\sigma}^{\mathcal{M}}] \quad (2.7)$$

$$\alpha_{\sigma}^{\mathcal{M}} = \sigma \alpha \quad (2.8)$$

$$\top_{\sigma}^{\mathcal{M}} = TT \quad (2.9)$$

$$(S \times T)_{\sigma}^{\mathcal{M}} = P[S_{\sigma}^{\mathcal{M}}, T_{\sigma}^{\mathcal{M}}] \quad (2.10)$$

$$(S \rightarrow T)_{\sigma}^{\mathcal{M}} = E[S_{\sigma}^{\mathcal{M}}, T_{\sigma}^{\mathcal{M}}]. \quad (2.11)$$

2.3.1 Метатеоретические свойства НМ-Р

Здесь мы рассматриваем лишь некоторые базовые свойства системы, чтобы убедиться, что всё работает как ожидается: вычисления конfluenceнтны; типы сохраняются при любом вычислении; при необходимости все типы могут быть выведены; всё терминирует, и с точностью до редукции **let**-связываний это тот же самый STLC, с которого мы начали. Доказательства в основном используют стандартные приёмы, поскольку правила переинтерпретации устроены так, что их можно напрямую встроить в доказательства свойств STLC и системы типов Хиндли — Милнера.

Теорема 8 (Теорема Чёрча — Россера для НМ-Р). Рефлексивное транзитивное замыкание ($\rightarrow$) обладает свойством Чёрча — Россера.

Доказательство. Классическое доказательство через отношение параллельной редукции [31].

Параллельная редукция определяется следующим образом:

$$\begin{array}{c}
t \rightarrow t \qquad \frac{s \rightarrow s' \quad t \rightarrow t'}{(s, t) \rightarrow (s', t')} \qquad \frac{t \rightarrow t'}{\pi_i t \rightarrow \pi_i t'} \qquad \frac{t_1 \rightarrow t'_1 \quad t_2 \rightarrow t'_2}{\pi_i(t_1, t_2) \rightarrow t'_i} \\
\\
\frac{t \rightarrow t'}{\lambda x. t \rightarrow \lambda x. t'} \qquad \frac{s \rightarrow s' \quad t \rightarrow t'}{s t \rightarrow s' t'} \qquad \frac{s \rightarrow s' \quad t \rightarrow t'}{(\lambda x. s) t \rightarrow s'[x := t']} \\
\\
\frac{s \rightarrow s' \quad t \rightarrow t'}{\text{let } x = s \text{ in } t \rightarrow \text{let } x = s' \text{ in } t'} \qquad \frac{t \rightarrow t'}{\text{let } x = v \text{ in } t \rightarrow t'[x := v]}
\end{array}$$

□

Теорема 9 (Сохранение типов в НМ-Р). *Имеет место следующее:*

1. Если $\mathcal{M}$ — модель, $\gamma x : C[\Gamma^{\mathcal{M}}, \Gamma(x)^{\mathcal{M}}]$ для всех $x \in \Gamma$ и $\Gamma \vdash t : T$, то $\llbracket t \rrbracket^{\mathcal{M}} / \gamma : C[\Gamma^{\mathcal{M}}, T^{\mathcal{M}}]$.
2. Если $\Gamma \vdash s : S$ и $\Gamma, x : S, \Delta \vdash t : T$, то $\Gamma, \Delta \vdash t[x := s] : T$.
3. Если $\Gamma \vdash v : \sigma$ и $\Gamma, x : \sigma, \Delta \vdash t : T$, то $\Gamma, \Delta \vdash t[x := v] : T$.
4. Если $\Gamma \vdash t : T$ и $t \rightsquigarrow t'$, то $\Gamma \vdash t' : T$.

Доказательство. Структурная индукция по термам и по $(\rightsquigarrow)$. □

Теорема 10 (Сильная нормализуемость в НМ-Р). *Замкнутые термы, типизируемые в НМ-Р, сильно нормализуемы относительно $(\rightsquigarrow)$ и редуцируются до значения.*

Доказательство. Применяем метод логических отношений Тейта. Пусть SN — множество сильно нормализуемых термов, редуцируемых до значения. Рассмотрим отношение $\mathcal{R} \subseteq \text{PolyType} \times \text{Term}$:

- $t \in \mathcal{R}_\alpha, \mathcal{R}_\top$ тогда и только когда $t \in SN$;
- $t \in \mathcal{R}_{T_1 \times T_2}$ тогда и только когда $\pi_i t \in \mathcal{R}_{T_i}$ для $i = 1, 2$;
- $t \in \mathcal{R}_{S \rightarrow T}$ тогда и только когда для всех $s \in \mathcal{R}_S$ имеет место $ts \in \mathcal{R}_T$;
- $t \in \mathcal{R}_{\forall \vec{\alpha}. T}$ тогда и только когда t замкнут и $t \in \mathcal{R}_{T[\vec{\alpha} := \vec{s}]}$ для всех допустимых $\vec{s}$.

Заметим, что $\mathcal{R}_T \subseteq SN$. Более того, справедливо следующее свойство: если $t \in \mathcal{R}_{\forall \vec{\alpha}. T}$, то $\llbracket \text{nf}(t) \rrbracket^{\mathcal{M}} / \varepsilon \in \mathcal{R}_{[T] \mathcal{M}[\vec{\alpha} := \vec{s}]}$ для всех $\mathcal{M}$ таких, что все термы внутри него принадлежат соответствующим $\mathcal{R}_{\forall}$.

Тогда мы можем доказать нашу основную лемму: если $\Gamma \vdash t : T$ и γ — подстановка такая, что $\gamma x \in \mathcal{R}_{\Gamma(x)}$ для всех $x \in \Gamma$, то $\gamma t \in \mathcal{R}_T$. □

Теорема 11 (Консервативность над STLC). *Нормальные формы замкнутых термов, типизируемых в НМ-Р, типизируемы в STLC.*

Доказательство. Благодаря сохранению типов нормальная форма $\text{nf}(t)$ типизируема в НМ-Р. По определению, она является значением; следовательно, не содержит подтермов переинтерпретации и let-связываний, а значит, типизируема в STLC. □

Теорема 12. *Глобальный вывод типов в НМ-Р разрешим.*

Доказательство. Достаточно расширить любой алгоритм вывода типов на основе унификации для системы типов Хиндли — Милнера так, чтобы он поддерживал модели и переинтерпретацию.

- Поскольку модели задаются явным образом, достаточно лишь проверить их.
- Поскольку полиморфные функции не зависят от локального контекста, их тип полностью определяется в момент добавления в контекст.
- После проверки модели M в $x^{\mathcal{M}}$ можно напрямую вычислить $\llbracket T \rrbracket_{[\vec{\alpha} := \vec{\eta}]}^{\mathcal{M}}$ (так как $\mathcal{M}$ известно, а T полностью определена), где $\vec{\eta}$ — свежие метапеременные.

□

Таким образом, в этом разделе мы ввели корректную систему типов на основе системы типов Хиндли — Милнера, способную интерпретировать собственные замкнутые термы в произвольной “декартово замкнутой категории”, определяемой внутри неё. Эта система не является по-настоящему *рефлексивной*, поскольку модели не являются полностью внутренними: они требуют специальной формы операторов типов, которая недоступна остальной части системы типов. Чтобы интернализировать определение моделей, мы переходим в следующем разделе к системе с зависимыми типами.

2.4 Почти рефлексивный язык с зависимыми типами: MLTT-R

Теперь мы готовы взяться за главную задачу: спроектировать минимальный рефлексивный язык с зависимыми типами, а также предоставить в нём внутренние модели самого языка для полиморфизма по вселенным, интернализации внешней параметричности и возможности заниматься синтетической математикой внутри него. В качестве отправной точки мы берём теорию типов Мартина — Лёфа с Π -, Σ - и Id -типами, типом натуральных чисел и бесконечной некумулятивной иерархией универсов. Как обсуждалось в разделе 2.2, наилучшее практическое решение проблемы переинтерпретации здесь — запретить редукции под ней, переинтерпретировать термы как есть и ограничить модели с помощью Id -типов. Тем не менее, по сравнению с случаем без зависимых типов, возникает бесконечно больше проблем, если мы хотим определить по-настоящему рефлексивную систему:

1. Если мы наивно перенесём подход к типизации моделей из раздела 2.3, то для спецификации модели нам потребуется задать счётно много моделей, по одной на каждый универсум. И даже если мы это сделаем, к какому универсуму принадлежит тип моделей?
2. Теперь, когда у нас есть вычисления в типах, мы обязаны разобраться с правилом конверсии под переинтерпретацией:

$$\frac{t : T \quad T \equiv T'}{t : T'} \rightsquigarrow \frac{[t] : [T] \quad [T] \equiv [T']}{[t] : [T']}$$

Проблема в том, что после интерпретации $[T]$ лишь *пропозиционально* равно $[T']$.

3. Если мы хотим по-настоящему рефлексивную систему, нам нужно решить, что делать с *переинтерпретацией переинтерпретации*:

$$\lambda x. \llbracket [x]_1 \rrbracket_2 \quad \text{и} \quad \llbracket \lambda x. [x]_1 \rrbracket_2$$

В первом случае мы выполняем две последовательные интерпретации над ещё не известным x ; во втором — интерпретируем сам процесс интерпретации. Что это должно значить?

Чтобы справиться с проблемами размера универсумов, обратим внимание на следующее наблюдение: в системе без операторов связываний переменных по уровням каждый терм использует лишь конечное число уровней. Следовательно, чтобы переинтерпретировать такой терм, достаточно задать конечное число моделей универсов, которые совместно помещаются в некоторый более крупный универс конечного уровня. Полиморфность универсов не нужна! Однако теперь в суждениях о типизации нам приходится отслеживать максимальный уровень, встречающийся в терме.

Для интерпретации правила конверсии мы сначала определяем, в каких именно местах оно действительно используется; для этого *необходимо* использовать двустороннюю проверку типов. В момент конверсии интерпретация обладает достаточной информацией, чтобы знать как T , так и T' ; таким образом, мы можем перевести цепочку редукций, выполняемых при проверке дефинициального равенства, в цепочку пропозициональных равенств, взятых из условий согласованности, доказанных в определении модели. Затем точка конверсии обёртывается в элиминацию равенства с синтезированным свидетелем в качестве экзаменуемого терма.

При переинтерпретации переинтерпретации на самом деле мы имеем дело лишь с “двойной переинтерпретацией”: случаи вида $\lambda x. \llbracket [x]_1 \rrbracket_2$ можно обработать путём *слияния* двух моделей в новую, — техника напоминает *Collapsing towers of interpreters* [6]; в нашей системе это не требуется, поскольку самая внутренняя переинтерпретация в этом случае может выполнять вычисления. Что касается интерпретации самой интерпретации, как в $\llbracket \lambda x. \llbracket [x]_1 \rrbracket_2 \rrbracket$, мы явно запрещаем такие конструкции, вводя семейство модальностей $\Box_\kappa$, означающее “при построении этого терма не использовалась интерпретация, а все встречающиеся в нём уровни не выше κ ”. Для переинтерпретации термы должны иметь тип с модальностью. В нотации модальности и общей дисциплине типизации мы в значительной мере следуем работе Gratzer и соавторов [18] с некоторыми важными отличиями, которые обсуждаются ниже. Мы также используем $\Box$ для ослабления запрета на вычисления внутри переинтерпретации, разрешая “вшивать” выражения с модальностью в другие выражения с модальностью; внутри вставок вычислениям разрешено приводить термы к модальному виду для дальнейшей интерпретации.

Грамматические правила, приведённые на рис. 2.4, отражают обсуждение выше. Обратим внимание, что тип моделей $\mathcal{M}$ аннотирован последовательностью уровней $\ell_0, \dots, \ell_\kappa$: уровень ℓ_j — это уровень, на котором оказываются типы из $\mathcal{U}_j$ после интерпретации в такой модели.

Правила типизации также являются прямой адаптацией общего подхода к двусторонней типизации систем с зависимыми типами [17] и приведены на рис. 2.5–2.6; правила размещения по уровням построены так, что $\Gamma \vdash_\ell \text{level}(T) = \kappa$ всегда влечёт $\kappa \leq \ell$.

Важное отличие от стандартного подхода двусторонней типизации, согласно которому “конструкторы проверяются, а элиминаторы выводятся”, состоит в том, что уровни типов всегда выводятся: у нас нет кумулятивности универсов и нет оператора подъёма, поэтому мы не можем проверять уровни Σ и Π . Также отметим, что мы явно задаём уровни, необходимые для проверки t в типовой переинтерпретации T^t : это происходит потому, что $\mathcal{M}$ дефинициально равно некоторому огромному вложенному Σ -типу, и сопоставление с ним было бы нецелесообразно, если бы он выводился из t .

Как уже было сказано, переинтерпретация ограничена действием на модальные типы; кроме того, мы требуем незафиксированный контекст для переинтерпретации. В совокупности эти ограничения гарантируют, что модальные термы действительно не содержат

Context	Γ	$::=$	$\cdot$	пустой контекст
			$\Gamma, x : T$	расширение переменной
			$\Gamma. \ell \blacksquare$	фиксация контекста
Type	T, S	$::=$	$\mathcal{U}_\ell$	универсум
			$\text{El}_\ell(t)$	универсальное раскодирование
			Nat_ℓ	тип натуральных чисел
			$(x : S) \times T$	Σ -тип
			$(x : S) \rightarrow T$	Π -тип
			$t_1 =_T t_2$	тип тождества
			$\Box_\kappa T$	тип переинтерпретируемых термов
			$\mathcal{M}^{\ell_0, \dots, \ell_\kappa}$	тип моделей
			$T^t_{\ell_0, \dots, \ell_\kappa}$	типовая переинтерпретация
Level	ℓ, κ, j	$\in$	$\mathbb{N}$	конечные уровни
Term	t, s	$::=$	$x \mid y \mid z \mid \mathfrak{m} \mid \dots$	переменные
			$\mathcal{U}_\ell$	код универсума
			Nat_ℓ	код типа натуральных чисел
			0	ноль
			St	преемник
			$\text{natind}_{x.T} t \{s_0, s_S\}$	индукция
			$\text{Sigma}(x : s)t$	код Σ -типа
			(s, t)	пара
			$\pi_1 t \mid \pi_2 t$	проекции
			$\text{Pi}(x : s)t$	код Π -типа
			$\lambda x. t$	абстракция
			ts	применение
			$\text{Id} t t_1 t_2$	код типа тождества
			refl	рефлексивность
			$J_{xy.S} t s$	правило J (элиминация типа равенства)
			$\text{Box}_\kappa t$	код модального типа
			$[t] \blacksquare$	фиксация
			$[t] \blacksquare$	снятие фиксации
			$\llbracket t \rrbracket^{\ell_0, \dots, \ell_\kappa}$	внутренняя интерпретация
			$t : T$	аннотация типа

Рис. 2.4: Грамматика MLTT-R

$$\begin{array}{c}
\text{WF-NIL} \quad \cdot \vdash_\ell \\
\text{WF-VAR} \quad \frac{\Gamma \vdash_\ell \quad \Gamma \vdash_\ell \text{level}(T) = \kappa}{\Gamma, x : T \vdash_\ell} \\
\text{WF-LOCK} \quad \frac{\Gamma \vdash_\ell \quad \blacksquare \notin \Gamma}{\Gamma. \ell \blacksquare \vdash_\kappa} \\
\text{LEVEL-EL} \quad \frac{\Gamma \vdash_\ell t \Leftarrow \mathcal{U}_\kappa}{\Gamma \vdash_\ell \text{level}(\text{El}_\kappa(t)) = \kappa} \\
\\
\text{LEVEL-UNI} \quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \text{level}(\mathcal{U}_\kappa) = \kappa + 1} (\kappa + 1 \leq \ell) \\
\text{LEVEL-NAT} \quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \text{level}(\text{Nat}_\kappa) = \kappa} (\kappa \leq \ell) \\
\\
\text{LEVEL-SIGMA, LEVEL-PI} \quad \frac{\Gamma \vdash_\ell \text{level}(S) = \kappa \quad \Gamma, x : S \vdash_\ell \text{level}(T) = j}{\Gamma \vdash_\ell \text{level}((x : S) \times T) = \text{level}((x : S) \rightarrow T) = \max(\kappa, j)} \\
\\
\text{LEVEL-ID} \quad \frac{\Gamma \vdash_\ell \text{level}(T) = \kappa \quad \Gamma \vdash_\ell t_1, t_2 \Leftarrow T}{\Gamma \vdash_\ell \text{level}(t_1 =_T t_2) = \kappa} \\
\text{LEVEL-BOX} \quad \frac{\blacksquare \notin \Gamma \quad \Gamma. \ell \blacksquare \vdash_\kappa \text{level}(T) = j}{\Gamma \vdash_\ell \text{level}(\Box_\kappa T) = \kappa} (\kappa \leq \ell) \\
\\
\text{LEVEL-MODEL} \quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \text{level}(\mathcal{M}_{\ell_0, \dots, \ell_\kappa}) = j} (j = \max(\ell_0, \dots, \ell_\kappa) + 2 \leq \ell) \\
\\
\text{LEVEL-TYINTERP} \quad \frac{\Gamma \vdash_\ell t \Leftarrow \mathcal{M}_{\ell_0, \dots, \ell_\kappa} \quad \blacksquare \notin \Gamma \quad \Gamma. \ell \blacksquare \vdash_\kappa \text{level}(T) = j}{\Gamma \vdash_\ell \text{level}(T_{\ell_0, \dots, \ell_\kappa}^t) = \ell_j}
\end{array}$$

Рис. 2.5: Правила корректности контекста и размещения по уровням

подтермов переинтерпретации. Важное отличие от MLTT $\blacksquare$ [18], помимо добавления семейства модальностей, состоит в способе фиксации контекста в $[_]_{\blacksquare}$ и снятия фиксации в $[_]_{\blacksquare}$: при фиксации мы требуем, чтобы внешний контекст был незафиксирован; при снятии фиксации, вместо удаления всех фиксаций, мы удаляем контекст до первой фиксации. Фактически, это накладывает ограничение, что контекст типизации никогда не содержит более одной фиксации. Таким образом, например, наш $\Box_\ell$ не является комонадой, но аксиома K модальной логики всё ещё выполняется¹:

$$\frac{\cdot \vdash_\ell \text{level}(A, B) = _}{\cdot \vdash_\kappa \lambda f x. [[f]_{\blacksquare} [x]_{\blacksquare}]_{\blacksquare} : \Box_\ell(A \rightarrow B) \rightarrow \Box_\ell A \rightarrow \Box_\ell B} (\ell \leq \kappa)$$

Таким образом, снятие фиксации по-настоящему приобретает семантику “вставки” в том смысле, что оно может содержать лишь модальные значения из “внешнего мира”.

2.4.1 Правила вычисления MLTT-R и модели систем с зависимыми типами

Заметим, что приведённое выше обсуждение не зависело от выбора формализма, в котором излагаются модели. Теперь, однако, пришло время зафиксировать формализм, поскольку без него невозможно дать определение того, куда вычисляется $\mathcal{M}$.

¹Интересно, что аналогичная K-модальность впервые описана Birkedal и соавторами в работе “Modal dependent type theory and dependent right adjoints” [9]. Совпадение: они тоже уточняют суждения о типизации уровнями универсов! Однако у них лишь одна модальность.

$$\begin{array}{c}
\text{INFER-UNI} \quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \mathbf{U}_\kappa \Rightarrow \mathcal{U}_{\kappa+1}} (\kappa + 1 \leq \ell) \quad \text{INFER-NAT} \quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell \mathbf{Nat}_\kappa \Rightarrow \mathcal{U}_\kappa} (\kappa \leq \ell) \quad \text{CHECK-ZERO} \quad \frac{\Gamma \vdash_\ell}{\Gamma \vdash_\ell 0 \Leftarrow \mathbf{Nat}_\kappa} \quad \text{CHECK-SUCC} \quad \frac{\Gamma \vdash_\ell t \Leftarrow \mathbf{Nat}_\kappa}{\Gamma \vdash_\ell S t \Leftarrow \mathbf{Nat}_\kappa} \\
\\
\text{INFER-NATIND} \quad \frac{\Gamma \vdash_\ell t \Rightarrow \mathbf{Nat}_\kappa \quad \Gamma, x : \mathbf{Nat}_\kappa \vdash_\ell \text{level}(T) = _ \quad \Gamma \vdash_\ell s_0 \Leftarrow T[x := 0 : \mathbf{Nat}_\kappa] \quad \Gamma \vdash_\ell s_S \Leftarrow (y : \mathbf{Nat}_\kappa) \rightarrow (z : T[x := y]) \rightarrow T[x := S y : \mathbf{Nat}_\kappa]}{\Gamma \vdash_\ell \text{natind}_{x.T} t \{s_0, s_S\} \Rightarrow T[x := t]} \\
\\
\text{INFER-SIGMA} \quad \frac{\Gamma \vdash_\ell s \Rightarrow \mathcal{U}_\kappa \quad \Gamma, x : \text{El}_\kappa(s) \vdash_\ell t \Rightarrow \mathcal{U}_j}{\Gamma \vdash_\ell \mathbf{Sigma}(x : s)t \Rightarrow \mathcal{U}_{\max(\kappa, j)}} \quad \text{CHECK-PAIR} \quad \frac{\Gamma \vdash_\ell s \Leftarrow S \quad \Gamma \vdash_\ell t \Leftarrow T[x := s : S]}{\Gamma \vdash_\ell (s, t) \Leftarrow (x : S) \times T} \\
\\
\text{INFER-Proj}_1 \quad \frac{\Gamma \vdash_\ell t \Rightarrow (x : S) \times T}{\Gamma \vdash_\ell \pi_1 t \Rightarrow S} \quad \text{INFER-Proj}_2 \quad \frac{\Gamma \vdash_\ell t \Rightarrow (x : S) \times T}{\Gamma \vdash_\ell \pi_2 t \Rightarrow T[x := \pi_1 t]} \quad \text{INFER-PI} \quad \frac{\Gamma \vdash_\ell s \Rightarrow \mathcal{U}_\kappa \quad \Gamma, x : \text{El}_\kappa(s) \vdash_\ell t \Rightarrow \mathcal{U}_j}{\Gamma \vdash_\ell \mathbf{Pi}(x : s)t \Rightarrow \mathcal{U}_{\max(\kappa, j)}} \\
\\
\text{CHECK-ABS} \quad \frac{\Gamma, x : S \vdash_\ell t \Leftarrow T}{\Gamma \vdash_\ell \lambda x. t \Leftarrow (x : S) \rightarrow T} \quad \text{INFER-APP} \quad \frac{\Gamma \vdash_\ell t \Rightarrow (x : S) \rightarrow T \quad \Gamma \vdash_\ell s \Leftarrow S}{\Gamma \vdash_\ell t s \Rightarrow T[x := s : S]} \\
\\
\text{INFER-ID} \quad \frac{\Gamma \vdash_\ell t \Rightarrow \mathcal{U}_\kappa \quad \Gamma \vdash_\ell t_1, t_2 \Leftarrow \text{El}_\kappa(t)}{\Gamma \vdash_\ell \mathbf{Id} t t_1 t_2 \Rightarrow \mathcal{U}_\kappa} \quad \text{CHECK-REFL} \quad \frac{\Gamma \vdash_\ell t_1 \equiv t_2 : T}{\Gamma \vdash_\ell \text{refl} \Leftarrow t_1 =_T t_2} \\
\\
\text{INFER-J} \quad \frac{\Gamma \vdash_\ell t \Rightarrow t_1 =_T t_2 \quad \Gamma, x : T, y : t_1 =_T x \vdash_\ell \text{level}(S) = _ \quad \Gamma \vdash_\ell s \Leftarrow S[x := t_1 : T, y := \text{refl} : t_1 =_T t_1]}{\Gamma \vdash_\ell \mathbf{J}_{xy.S} t s \Rightarrow S[x := t_2 : T, y := t]} \\
\\
\text{INFER-VAR} \quad \frac{\blacksquare \notin \Delta \quad \Gamma, x : T, \Delta \vdash_\ell}{\Gamma, x : T, \Delta \vdash_\ell x \Rightarrow T} \quad \text{INFER-BOX} \quad \frac{\blacksquare \notin \Gamma \quad \Gamma. \ell \blacksquare \vdash_\kappa t \Rightarrow \mathcal{U}_j}{\Gamma \vdash_\ell \mathbf{Box}_\kappa t \Rightarrow \mathcal{U}_j} \quad \text{CHECK-LOCK} \quad \frac{\blacksquare \notin \Gamma \quad \Gamma. \ell \blacksquare \vdash_\kappa t \Leftarrow T}{\Gamma \vdash_\ell [t]_\blacksquare \Leftarrow \square_\kappa T} \\
\\
\text{INFER-UNLOCK} \quad \frac{\blacksquare \notin \Delta \quad \Gamma \vdash_\kappa t \Rightarrow \square_j T}{\Gamma. \kappa \blacksquare, \Delta \vdash_\ell [t]_\blacksquare \Rightarrow T} \quad \text{INFER-INTERP} \quad \frac{\Gamma \vdash_\ell t \Rightarrow \square_\kappa T \quad \blacksquare \notin \Gamma}{\Gamma \vdash_\ell \llbracket t \rrbracket^{\ell_0, \dots, \ell_\kappa} \Rightarrow (\mathfrak{m} : \mathcal{M}_{\ell_0, \dots, \ell_\kappa}) \rightarrow T^\mathfrak{m}} \\
\\
\text{INFER-ASCR} \quad \frac{\Gamma \vdash_\ell \text{level}(T) = _ \quad \Gamma \vdash_\ell t \Leftarrow T}{\Gamma \vdash_\ell t : T \Rightarrow T} \quad \text{CHECK-INFERRED} \quad \frac{\Gamma \vdash_\ell t \Rightarrow S \quad \Gamma \vdash_\ell S \equiv T}{\Gamma \vdash_\ell t \Leftarrow T}
\end{array}$$

Рис. 2.6: Правила двусторонней типизации

В соответствии с современным уровнем развития области мы используем *категории с семействами* [11] (или, сокращённо, CwF), поскольку это в настоящее время один из наиболее удобных формализмов для спецификации моделей теорий зависимых типов. Ниже приведено самодостаточное определение категории с семействами, которое даётся в явном виде для удобства при последующей интернализации:

Определение 45 (Категория). *Категория представляет собой следующую структуру²:*

1. Множество объектов Ob и семейство морфизмов $\text{Hom}(S, T)$, индексированное парами $S, T \in \text{Ob}$, причём $f \in \text{Hom}(S, T)$ обозначается как $f : S \rightarrow T$. Кроме того, объект S называется источником морфизма f , а T — целью морфизма f .
2. Для каждого $A \in \text{Ob}$ существует тождественный морфизм $\text{id}_A : A \rightarrow A$; для каждого $f : A \rightarrow B$ и $g : B \rightarrow C$ существует их композиция $g \circ f : A \rightarrow C$. Если источник и цель тождественного морфизма ясны из контекста, подстрочный индекс опускается. Кроме того, выполняются следующие свойства:

$$\text{id} \circ f = f = f \circ \text{id} \qquad (f \circ g) \circ h = f \circ (g \circ h)$$

для произвольных морфизмов f, g, h с совместимыми источниками и целями.

Определение 46 (Терминальный объект). Терминальным объектом категории $\mathcal{C}$ называется объект $\top \in \text{Ob}_{\mathcal{C}}$ такой, что для каждого $A \in \text{Ob}_{\mathcal{C}}$ существует **единственный** морфизм $!_A : A \rightarrow \top$.

Определение 47 (Предпучок). Предпучком над категорией $\mathcal{C}$ (обозначается $P : \mathcal{C}^{\text{op}} \rightarrow \text{Set}$) называется отображение, которое объектам категории $\mathcal{C}$ ставит в соответствие множества, а морфизмам $\mathcal{C}$ — функции между множествами, такое, что если $f : A \rightarrow_{\mathcal{C}} B$, то $P(f)$ является функцией из $P(B)$ в $P(A)$. При этом выполняются следующие свойства:

$$P(\text{id}_A) = \text{id}_{P(A)} \qquad P(g \circ f) = P(f) \circ P(g)$$

где $\text{id}_{P(A)}$ — тождественная функция на множестве $P(A)$, а $P(f) \circ P(g)$ — композиция функций $P(f)$ и $P(g)$.

Определение 48 (Категория элементов). Пусть $P : \mathcal{C}^{\text{op}} \rightarrow \text{Set}$ — предпучок. Категорией элементов $\int_{\mathcal{C}} P$ называется категория, объекты которой являются парами (A, x) , где $A \in \text{Ob}_{\mathcal{C}}$ и $x \in P(A)$, а морфизмы из (A, x) в (B, y) — это морфизмы $f : A \rightarrow_{\mathcal{C}} B$ такие, что $P(f)(y) = x$.

Определение 49 (Категория с семействами). Категория с семействами — это следующая структура:

1. Категория контекстов $\mathcal{C}$, являющаяся категорией с терминальным объектом;
2. Предпучок типов $\text{Ty} : \mathcal{C}^{\text{op}} \rightarrow \text{Set}$. Для данного морфизма $\gamma : \Gamma \rightarrow_{\mathcal{C}} \Delta$ и типа $T \in \text{Ty}(\Delta)$ элемент $\text{Ty}(\gamma)(T) \in \text{Ty}(\Gamma)$ обычно записывают в сокращённой форме $T[\gamma]$;
3. Предпучок термов $\text{Tm} : (\int_{\mathcal{C}} \text{Ty})^{\text{op}} \rightarrow \text{Set}$. Аналогично случаю с типами, $\text{Tm}(\gamma)(t) \in \text{Tm}(\Gamma, T)$ обычно записывают в сокращённой форме $t[\gamma]$;

²Разумеется, декартово замкнутая категория (41) тоже является категорией. Часть определений повторяется для самодостаточности.

4. Для каждого $\Gamma \in \text{Ob}_C$ и $T \in \text{Ty}(\Gamma)$ существует расширенный контекст $\Gamma.T \in \text{Ob}_C$ вместе с морфизмом ослабления $p : \Gamma.T \rightarrow \Gamma$ и элементом переменной $q : \text{Tm}(\Gamma.T, T[p])$. Кроме того, для любого Δ , $\gamma : \Delta \rightarrow \Gamma$ и $t \in \text{Tm}(\Delta, T[\gamma])$ существует единственная расширенная подстановка $\langle \gamma, t \rangle : \Delta \rightarrow \Gamma.T$, удовлетворяющая соотношениям:

$$p \circ \langle \gamma, t \rangle = \gamma \qquad q[\langle \gamma, t \rangle] = t$$

Определение CwF (опр. 49) в явном виде выражает базовую структуру теории зависимых типов: объекты категории $\mathcal{C}$ представляют контексты, а морфизмы — подстановки, отображающие один контекст в другой. Подстановка действует как на типы, так и на термы; если в контексте Γ задан корректный тип T , можно расширить контекст, получив новый контекст $\Gamma, x:T$, в котором существует корректно типизированный терм $x:T$; подстановки в расширенный контекст строятся путём задания подходящего для x терма.

Это базовое определение легко расширяется добавлением образователей типов (и терминов) для любой системы. Соответственно, CwF для MLTT-R должна включать Ty- и Tm-формы для аналогов универсумов, Π - и Σ -типов, натуральных чисел и типов тождества. Интерпретация для $\Box_\ell$ не требуется, поскольку мы успешно предотвратили появление типов в боксах внутри термов в боксах благодаря жёсткой дисциплине $\blacksquare$ -ирования³. Таким образом, контексты, требующие интерпретации, строятся путём стандартных расширений из пустого контекста, и обычная CwF достаточна для наших целей. Поскольку фактическое интернализированное определение нашей CwF в значительной мере повторяет правила типизации и вычисления с рисунков 2.5–2.6 и 2.7–2.8, но уже на внутреннем языке, мы не приводим его здесь; для получения более подробных сведений см. наши дополнительные материалы.

Перейдём, наконец, к правилам вычисления, приведённым на рисунках 2.7–2.8 (полное отношение вычисления типов записывается как $\Gamma \vdash_\ell T \rightsquigarrow T'$, а вычисление термов — как $\Gamma \vdash_\ell t \rightsquigarrow t' \Leftrightarrow T$; если тип не важен, а контекст или ограничение на уровень просто наследуются, они опускаются ради краткости). Благодаря интернализированному определению CwF мы знаем, как вычисляется $\mathcal{M}$; остальные правила вычисления типов и термов в большинстве своём являются стандартными правилами вычисления MLTT для двусторонней типизации (пока что без η -равенства). Важно, что мы **не выполняем вычисление внутри заблокированных термов и внутри типов в боксах**, чтобы не попасть вновь под действие варианта нашей теоремы невозможности 7 (модальный вариант изложен в разделе обсуждения, см. раздел 2.5.2).

Помимо этого, необходимо также привести правила вычисления реинтерпретации; как оказалось, *прямая реинтерпретация терминов как есть* позволяет определить реинтерпретацию не как редукс, а как кипу тотальных функций на типах и терминах — так же, как и в случае без зависимых типов (раздел 2.3). Если быть точным, нам необходимы две взаимно рекурсивные функции для термов: $\llbracket t \rrbracket_{\leftarrow T}^\Gamma$ вычисляется для термов в режиме проверки, а $\llbracket t \rrbracket_{\Rightarrow \mathfrak{m}}^\Gamma$ — для термов в режиме синтеза и возвращает выведенный тип; обе вычисляют соответствующие элементы $\text{Tm}(\Gamma, _)$ из CwF $\mathfrak{m}$. Выражение $T_{\mathfrak{m}}^\Gamma$ вычисляет элемент $\text{Ty}(\Gamma)$. Для обработки правила конверсии требуется ещё одна вспомогательная функция, вычисляющая цепочку равенств; у неё есть две тонкости:

- На данном этапе мы ещё не знаем, является ли система сильно нормализующейся, поэтому потенциально нам может встретиться бесконечная цепочка равенств.
- В системах, где равенство существенно зависит от доказательства (и мы хотели бы иметь возможность развивать теорию именно в этом направлении), различные цепочки редукций могут приводить к различным свидетелям равенства, так что такая функция потенциально может быть некорректно определённой.

³Это же и есть причина, по которой мы называем данную систему *почти* рефлексивной: её внутренняя реинтерпретация не покрывает модальностей и саму реинтерпретацию.

$$\begin{array}{c}
\frac{t \rightsquigarrow t' \Leftarrow \mathcal{U}_\kappa}{\text{El}_\kappa(t) \rightsquigarrow \text{El}_\kappa(t')} \quad \text{El}(\mathbf{U}_\kappa) \rightsquigarrow \mathcal{U}_\kappa \quad \text{El}(\mathbf{Nat}_\kappa) \rightsquigarrow \text{Nat}_\kappa \quad \text{El}(\mathbf{Sigma}(x : s)t) \rightsquigarrow (x : \text{El}(s)) \times \text{El}(t) \\
\\
\text{El}(\mathbf{Pi}(x : s)t) \rightsquigarrow (x : \text{El}(s)) \rightarrow \text{El}(t) \quad \text{El}(\mathbf{Id} \, t \, t_1 \, t_2) \rightsquigarrow t_1 =_{\text{El}(t)} t_2 \quad \text{El}(\mathbf{Box}_\kappa t) \rightsquigarrow \Box_\kappa \text{El}(t) \\
\\
\frac{S \rightsquigarrow S'}{(x : S) \times T \rightsquigarrow (x : S') \times T} \quad \frac{\Gamma, x : S \vdash_\ell T \rightsquigarrow T'}{\Gamma \vdash_\ell (x : S) \times T \rightsquigarrow (x : S) \times T'} \quad \frac{S \rightsquigarrow S'}{(x : S) \rightarrow T \rightsquigarrow (x : S') \rightarrow T} \\
\\
\frac{\Gamma, x : S \vdash_\ell T \rightsquigarrow T'}{\Gamma \vdash_\ell (x : S) \rightarrow T \rightsquigarrow (x : S) \rightarrow T'} \quad \frac{T \rightsquigarrow T'}{t_1 =_T t_2 \rightsquigarrow t_1 =_{T'} t_2} \quad \frac{t_1 \rightsquigarrow t'_1 \Leftarrow T}{t_1 =_T t_2 \rightsquigarrow t'_1 =_T t_2} \\
\\
\frac{t_2 \rightsquigarrow t'_2 \Leftarrow T}{t_1 =_T t_2 \rightsquigarrow t_1 =_T t'_2} \quad \mathcal{M}_{\ell_0, \dots, \ell_\kappa} \rightsquigarrow \text{modelDef}(\ell_0, \dots, \ell_\kappa) \quad T_{\ell_0, \dots, \ell_\kappa}^t \rightsquigarrow t. \text{Tm}(t. \cdot, T_t)
\end{array}$$

Рис. 2.7: Вычисления на типах MLTT-R

$$\begin{array}{c}
\frac{t \rightsquigarrow t'}{St \rightsquigarrow St'} \quad \frac{t \rightsquigarrow t'}{\text{natind}_{x.T} t \{s_0, s_S\} \rightsquigarrow \text{natind}_{x.T} t' \{s_0, s_S\}} \\
\\
\text{natind}_{x.T}(0 : \text{Nat}_\ell) \{s_0, s_S\} \rightsquigarrow (s_0 : T[x := 0 : \text{Nat}_\ell]) \\
\\
\text{natind}_{x.T}(St : \text{Nat}_\ell) \{s_0, s_S\} \rightsquigarrow (s_S : (x : \text{Nat}_\ell) \rightarrow T \rightarrow T[x := Sx]) t (\text{natind}_{x.T}(t : \text{Nat}_\ell) \{s_0, s_S\}) \\
\\
\frac{s \rightsquigarrow s'}{\mathbf{Sigma}(x : s)t \rightsquigarrow \mathbf{Sigma}(x : s')t} \quad \frac{\Gamma, x : \text{El}(s) \vdash_\ell t \rightsquigarrow t'}{\Gamma \vdash_\ell \mathbf{Sigma}(x : s)t \rightsquigarrow \mathbf{Sigma}(x : s)t'} \quad \frac{s \rightsquigarrow s'}{(s, t) \rightsquigarrow (s', t)} \\
\\
\frac{t \rightsquigarrow t'}{(s, t) \rightsquigarrow (s, t')} \quad \frac{t \rightsquigarrow t'}{\pi_i t \rightsquigarrow \pi_i t'} \quad \pi_i(t_1, t_2) \rightsquigarrow t_i \quad \frac{s \rightsquigarrow s'}{\mathbf{Pi}(x : s)t \rightsquigarrow \mathbf{Pi}(x : s')t} \\
\\
\frac{\Gamma, x : \text{El}(s) \vdash_\ell t \rightsquigarrow t'}{\Gamma \vdash_\ell \mathbf{Pi}(x : s)t \rightsquigarrow \mathbf{Pi}(x : s)t'} \quad \frac{\Gamma, x : S \vdash_\ell t \rightsquigarrow t' : T}{\Gamma \vdash_\ell \lambda x. t \rightsquigarrow \lambda x. t'} \quad \frac{t \rightsquigarrow t'}{ts \rightsquigarrow t' s} \quad \frac{s \rightsquigarrow s'}{ts \rightsquigarrow t s'} \\
\\
\frac{\Gamma \vdash_\ell s \Rightarrow S}{(\lambda x. t)s \rightsquigarrow t[x := s : S]} \quad \frac{t \rightsquigarrow t'}{\mathbf{Id} \, t \, t_1 \, t_2 \rightsquigarrow \mathbf{Id} \, t' \, t_1 \, t_2} \quad \frac{t_1 \rightsquigarrow t'_1}{\mathbf{Id} \, t \, t_1 \, t_2 \rightsquigarrow \mathbf{Id} \, t \, t'_1 \, t_2} \quad \frac{t_2 \rightsquigarrow t'_2}{\mathbf{Id} \, t \, t_1 \, t_2 \rightsquigarrow \mathbf{Id} \, t \, t_1 \, t'_2} \\
\\
\frac{t \rightsquigarrow t'}{\mathbf{J}_{xy.S} t \, s \rightsquigarrow \mathbf{J}_{xy.S} t' \, s} \quad \mathbf{J}_{xy.S}(\text{refl} : t_1 =_T t_2) \, s \rightsquigarrow (s : S[x := t_2, y := \text{refl} : t_1 =_T t_2]) \\
\\
\frac{t \rightsquigarrow t'}{(t : T) \rightsquigarrow (t' : T)} \quad \frac{t \rightsquigarrow t'}{\llbracket t \rrbracket^{\ell_0, \dots, \ell_\kappa} \rightsquigarrow \llbracket t' \rrbracket^{\ell_0, \dots, \ell_\kappa}} \quad \llbracket [t]_{\blacksquare} : \Box_\kappa T \rrbracket^{\ell_0, \dots, \ell_\kappa} \rightsquigarrow \lambda \mathbf{m}. \llbracket [t] \rrbracket_{\Leftarrow T} \mathbf{m}
\end{array}$$

Рис. 2.8: Вычисления на термах MLTT-R

Наше *хитрое* решение состоит в следующем. Зафиксируем определённую стратегию вычисления (например, строгую) и используем её как при двусторонней проверке типов, так и при выполнении функции извлечения свидетелей. Если мы запускаем функцию извлечения свидетелей на терме, который был успешно типизирован алгоритмом типизации, то этот алгоритм уже завершил работу, а значит, и функция извлечения свидетелей также гарантированно завершится. Поскольку стратегия вычисления зафиксирована, функция извлечения свидетелей всегда будет выдавать один и тот же результат на одном и том же входе и, следовательно, является корректно определённой. И ещё раз: поскольку функции интерпретации непосредственно переводят правила двусторонней типизации термов в применения соответствующих конструкций модели, их явное определение не приводится.

2.4.2 Метатеоретические свойства MLTT-R

Ещё раз, мы ограничиваемся проверкой лишь самых существенных свойств: конфлюэнтностью вычислений, сохранением типов при вычислениях, завершаемостью всех вычислений, а также тем, что получившаяся система типов является консервативным расширением MLTT.

Теорема 13 (Теорема Чёрча — Россера для MLTT-R). *Рефлексивное транзитивное замыкание отношения ($\rightsquigarrow$) обладает свойством Чёрча — Россера.*

Доказательство. Классическое доказательство через отношение параллельной редукции [31]. Поскольку интерпретация вычисляется как рекурсивная функция на открытых термах, она коммутирует с подстановкой, что является необходимым условием конфлюэнтности. $\square$

Теорема 14 (Сохранение типов для MLTT-R). *Если $\Gamma \vdash t \Leftarrow T$ и $t \rightsquigarrow t'$, то $\Gamma \vdash t' \Leftarrow T$; аналогично для $\Gamma \vdash t \Rightarrow T$.*

Доказательство. Структурной индукцией по термам и ($\rightsquigarrow$). Заметим, что в двусторонней системе следует аккуратно выстроить правила так, чтобы подстановки применялись к термам в режиме вывода, чтобы выводимые термины оставались выводимыми, а проверяемые — проверяемыми. Аналогично доказательству сохранения типов для HM-R (раздел 2.3.1), выполняется ключевая лемма:

$$\frac{\Gamma.\ell \blacksquare, \Delta \vdash_{\kappa} t \Leftarrow T \quad \Gamma \vdash_{\ell} s \Leftarrow \mathcal{M}^{\ell_0, \dots, \ell_{\kappa}}}{\Gamma \vdash_{\ell} \llbracket t \rrbracket_{\Leftarrow T} \Rightarrow s.\text{Tm}(s[\Delta], T_s^{\Delta})}$$

а также аналогичные леммы для $\llbracket _ \rrbracket_{\Rightarrow}$ и $_^{\Delta}_s$. $\square$

Теорема 15 (Тождественная модель). *Для каждого ℓ существует тождественная модель $I_{\ell} : \mathcal{M}_{0, \dots, \ell}$ со свойством $\llbracket [t]_{\blacksquare} \rrbracket I_{\ell} \equiv t$ для любого замкнутого t .*

Доказательство. В качестве типа контекстов возьмём универсум на соответствующем уровне; пусть $\text{Ty}_{\ell}(\Gamma) = \Gamma \rightarrow \mathcal{U}_{\ell}$; $\text{Tm}_{\ell}(\Gamma, T) = (\gamma : \Gamma) \rightarrow T \gamma$. Все остальные интерпретации термов определим так, чтобы они реинтерпретировались в те же самые термы. $\square$

Теорема 16 (Сильная нормализация для MLTT-R). *Замкнутые термины, типизируемые в MLTT-R, сильно нормализуются относительно ($\rightsquigarrow$).*

Доказательство. Достаточно любого доказательства нормализации для MLTT. Для методов, основанных на логических соотношениях Тейта (как было сделано для HM-R в разделе 2.3.1), положим, что $t \in \mathcal{R}_{\square_{\kappa} T}$ означает: «интерпретация t принадлежит $\mathcal{R}$ для интерпретации T для всех внутренних моделей из $\mathcal{R}_{\mathcal{M}}$ ». Для методов нормализации путём вычисления (как в [1]) возьмём значение такого t равным его функции интерпретации; чтобы прочитать значение обратно в терм, применим эту функцию к тождественной модели. $\square$

Теорема 17 (Консервативность относительно MLTT). *Для контекста Γ MLTT, типа T MLTT и терма t MLTT-R, если $\Gamma \vdash t \Leftarrow T$, то $\Gamma \vdash_{\text{MLTT}} \text{nf}(t) \Leftarrow T$.*

Доказательство. В силу сохранения типов нормальная форма $\text{nf}(t)$ типизируется в MLTT-R. Поскольку Γ не содержит ни боксов, ни блокировок, любой подтерм типа $\Box_{\kappa} T$ имеет вид $[_]_{\blacksquare}$. Так как T — тип MLTT, каждая такая блокировка обёрнута либо в $[_]_{\blacksquare}$, либо в $[_]_{\blacksquare}$. Во втором случае возникает редекс, что невозможно в нормальной форме; $[_]_{\blacksquare}$ возможен только в заблокированном контексте, а значит, выше него находится ещё одна блокировка $[_]_{\blacksquare}$. Следовательно, $\text{nf}(t)$ не содержит подтермов типа $\Box_{\kappa} T$, а значит, не содержит и подтермов разблокировки и интерпретации: это терм MLTT, типизируемый в двустороннем MLTT (который получается из MLTT-R путём удаления правил, связанных с модальностями, блокировкой и реинтерпретацией). $\square$

2.4.3 Полезные реинтерпретации внутри MLTT-R

Ниже мы приводим некоторые возможности языка, которых можно добиться за счёт реинтерпретации без какого-либо дополнительного расширения языка.

Примитивная, но эффективная стратификация

По Макбрайду [24, 25], техника примитивной, но эффективной стратификации работает следующим образом: вместо предоставления полной поддержки полиморфизма уровней вводим *оператор смещения*, который, получив терм t , использующий универсумы $U_i, U_j, U_k, \dots$, порождает терм, равный t за исключением того, что универсумы заменены на $U_{i+1}, U_{j+1}, U_{k+1}, \dots$. Смещённый терм также корректно типизирован, однако определяется «на один уровень выше». Хотя его нельзя непосредственно определить как единый оператор внутри MLTT-R, для каждого κ, ℓ можно определить операторы смещения $\uparrow_{\ell}^{\kappa} : \mathcal{M}_{\kappa, \kappa+1, \dots, \kappa+\ell}$, которые сдвигают на κ уровней вверх закоробленное определение, использующее универсумы вплоть до $\mathcal{U}_{\ell}$. В этой модели базовые предпучки категории контекстов смещены на κ по сравнению с тождественной моделью; все остальные интерпретации термов совпадают с определениями в тождественной модели.

Немного подробностей об определении $\uparrow_{\ell}^{\kappa}$ приведено в дополнительных материалах.

Внутренняя внешняя параметричность

Как показано Рейнольдсом [30], параметричность — это свойство системы типов, которое можно выразить простыми словами так: «полиморфная функция не может существенным образом зависеть от своих параметров». Таким образом, любая полиморфная функция типа σ удовлетворяет некоторой «свободной теореме» $\llbracket \sigma \rrbracket$, определяемой посредством реляционной интерпретации типа.

В недавних работах [7, 5] предложено упрощённое определение интерпретации параметричности для систем с зависимыми типами, названное «внешней параметричностью». И вновь: эта реляционная интерпретация определима в нашей системе *внутренним* образом с помощью реляционной модели $\text{Para}_{\ell} : \mathcal{M}_{1, \dots, \ell+1}$. Смещение на один уровень обусловлено тем, что тип $T : \mathcal{U}_{\ell}$ в такой модели интерпретируется тройкой $(T_1 \ T_2 : \mathcal{U}_{\ell}) \times (T_1 \rightarrow T_2 \rightarrow \mathcal{U}_{\ell})$. Поскольку интерпретируемые функции не зависят от локального контекста, мы обходим сложности с реляционным контекстом, отмеченные Альтенкирхом и соавторами [5]; в результате интерпретация даёт полноценное доказательство свободной теоремы.

Немного подробностей об определении Para_{ℓ} приведено в дополнительных материалах.

Синтетическая математика

Как подробно обсуждалось в недавней статье Навруцкого и соавторов [27], благодаря тому, что MLTT служит внутренним языком определённых $(\infty, 1)$ -категорий, в нём можно вести *синтетическую математику*: доказать внутри MLTT теорему о типах, интерпретировать её в модели MLTT и тем самым получить доказательство об объектах этой модели. Легко видеть, что оператор реинтерпретации, имеющийся в MLTT-R, позволяет вести синтетическую математику непосредственно в ней и сразу применять результаты, интерпретированные в подходящей модели.

2.5 Обсуждение и связанные работы

2.5.1 Что было сделано до нас

Первоначальная работа Ламбека [23] задала тон последующим исследованиям, установив соответствие между системами типов, основанными на лямбда-исчислении, и различными видами категорий, начиная с классического соответствия между просто типизированным лямбда-исчислением и декартово замкнутыми категориями.

В статье Конола Эллиота «Компиляция в категории» [16] показано, как с помощью плагина компилятора GHC интерпретировать (часть) Haskell в произвольной категории (опять же, определяемой в Haskell). Эта статья является прямым источником вдохновения для данной работы. Впрочем, есть важное различие: реинтерпретация в наших системах типов является *внутренней*, что открывает новые способы её использования, в частности (но не только) полиморфизм универсумов, внешнюю параметричность и многое другое. Вместе с тем подход Конола более гибок: его трансляция в категории требует от целевой категории реализации лишь тех примитивов, которые фактически используются при трансляции данного конкретного терма.

В работе Конора Макбрайда «Примитивная, но эффективная стратификация» [24, 25] (систематическое изложение этой идеи дано Фавонией и соавторами [20]) предложена прорывная идея использования *смещения* (displacement) как лёгкой формы полиморфизма универсумов: чтобы повторно использовать термин, типизированный на более низком уровне универсумов, достаточно поднять все уровни в терме, добавив постоянное смещение s . С точки зрения, продвигаемой нами, это можно рассматривать как реинтерпретацию выражения во внутренней модели системы типов, в которой универсум на уровне i интерпретируется как универсум на уровне $i + s$ (раздел 2.4.3).

В статье Жана-Филиппа Бернарди, Патрика Йанссона и Росса Патерсона «Параметричность и зависимые типы» [7] авторы предлагают прямую процедуру трансляции, позволяющую получать теоремы и доказательства параметричности из полиморфных определений в системе с зависимыми типами. Однако определение «внешней параметричности», которому мы следуем, дано Торстеном Альтенкирхом, Йорго Шамоуном, Амбрусом Капоши и Майклом Шульманом в статье «Внутренняя параметричность без интервала» [5]. И вновь мы можем переформулировать предложенную ими трансляцию как реинтерпретацию во внутренней модели. Альтенкирх и соавторы указали на проблему данного подхода: если применить внешнюю трансляцию к суждению с непустым контекстом, контекст тоже становится параметрическим; именно поэтому они добавляют операцию `rel` и получают систему типов с вычислительными свойствами *высшей наблюдательной теории типов*. Мы обходим проблему с контекстом, поскольку реинтерпретация применяется только к термам, контекст которых тоже «реинтерпретируем» и не затрагивается трансляцией (раздел 2.4.3).

2.5.2 Что можно было сделать иначе

Просто использовать метапрограммирование

В недавней статье «Сертифицирующее средство интерактивных доказательств для синтетической математики в Lean» Войцех Навруцкий и соавторы [27] адаптировали подход «Компиляция в категории» [16] к системе Lean с зависимыми типами, используя категории с семействами для представления внутренних моделей и применяя мощные средства метапрограммирования Lean вместо плагина компилятора. В результате им удалось создать основу для ведения синтетической математики в Lean: доказательства, полученные в MLTT, применяются к более специфическим категориям. Таким образом, наши результаты в разделе 2.4.3 близки к результатам, показанным в их статье. Более того, их реализация уже встроена в практическую реализацию средства интерактивных доказательств. Тем не менее, наша система типов не использует метапрограммирование, поскольку реинтерпретация является внутренней для неё, и она корректно обрабатывает правило конверсии. К тому же акценты немного различаются: мы используем реинтерпретацию для выражения фундаментальных языковых возможностей.

Вычислительная модальность глобальных элементов

Поскольку в разделе 2.4 мы ввели семейство модальностей $\Box_\ell$ реинтерпретируемых терминов, можно было бы надеяться допустить вычисление внутри закоробленных терминов. Чуть обобщая, следующее определение пригодится далее:

Определение 50. Система типов с зависимыми типами и типом функций $(\rightarrow)$ обладает вычислительной модальностью глобальных элементов $\Box$, если существуют терм с одним пропуском nes_ и атомарный терм с двумя пропусками $K_$ такие, что допустимы следующие правила:

$$\frac{\cdot \vdash T \text{ type}}{\Gamma \vdash \Box T \text{ type}} \quad \frac{\cdot \vdash t : T}{\Gamma \vdash \text{nes } t : \Box T} \quad \frac{\Gamma \vdash f : \Box (S \rightarrow T) \quad \Gamma \vdash t : \Box S}{\Gamma \vdash K f t : \Box T}$$

а также допустима редукция: $K (\text{nes } \lambda x.t) (\text{nes } s) \rightsquigarrow^* \text{nes}(t[x := s])$.

За исключением вычислительного правила, эти свойства в точности совпадают со свойствами модальностей $\Box_\ell$ из исходного раздела 2.4. Интуиция за данной редукцией следующая: если мы применим «глобальную функцию» к «глобальному аргументу», то мы должны иметь возможность «встроить» это применение, — что в точности и делает данная редукция.

Тогда соответствующее ослабление вычислительной реинтерпретации можно определить следующим образом.

Определение 51 (Модальная вычислительная реинтерпретация). Оператор типов $F[_]$ называется модальной вычислительной реинтерпретацией типов, если он определён на закоробленных типах (если $\Gamma \vdash \Box T \text{ Type}$, то $\Gamma \vdash F[T] \text{ Type}$), сохраняет вычисления на типах (если $T \rightarrow T'$, то $F[T] \rightarrow^* F[T']$) и вычисляется как примитивно рекурсивная функция на нормальных формах типов. Аналогично, терм с пропуском $\llbracket _ \rrbracket$ называется модальной вычислительной реинтерпретацией термов, если он сохраняет вычисления на термах, вычисляется как примитивно рекурсивная функция на нормальных формах термов и, если $\Gamma \vdash t : \Box T$, то $\Gamma \vdash \llbracket t \rrbracket : F[T]$.

Возможно, такое определение реинтерпретации «правильно»? К сожалению, для этого случая выполняется теорема о невозможности, аналогичная наивной версии 7:

Теорема 18 (Теорема о невозможности модальной реинтерпретации). *В конфлюентной системе типов с модальной вычислительной реинтерпретацией для произвольных корректно типизированных s и t должно выполняться:*

$$\text{app}[\llbracket \text{nec } \lambda x.t \rrbracket][\llbracket \text{nec } s \rrbracket] \equiv \llbracket \text{nec}(t[x := s]) \rrbracket$$

где $\equiv$ — дефинициальное равенство, а app — результат действия $\llbracket _ \rrbracket$ на K .

Доказательство. На конкретном примере. Пусть заданы корректные типы $\cdot \vdash S, T$ type и корректно типизированные термы $\cdot \vdash s : S$ и $\cdot, x : S \vdash t : T$. Рассмотрим следующий терм:

$$\Gamma, x : \Box(S \rightarrow T), y : \Box S \vdash \llbracket K x y \rrbracket : F[T]$$

Поскольку K атомарен, терм под интерпретацией находится в нормальной форме, а значит весь термин вычисляется в $\text{app}[\llbracket x \rrbracket][\llbracket y \rrbracket]$. Подставив $\text{nec } \lambda x.t$ вместо x и $\text{nec } s$ вместо y , получим $\text{app}[\llbracket \text{nec } \lambda x.t \rrbracket][\llbracket \text{nec } s \rrbracket]$.

С другой стороны, если сначала выполнить подстановку, то получим $\Gamma \vdash \llbracket K(\text{nec } \lambda x.t)(\text{nec } s) \rrbracket : F[T]$. Поскольку модальная реинтерпретация сохраняет редукцию, этот термин редуцируется в $\llbracket \text{nec}(t[x := s]) \rrbracket$.

В конфлюентной системе типов с функциями многошаговая редукция переставляется с подстановкой, поэтому два термина дефинициально эквивалентны. $\square$

Разумеется, это не запрещает нам использовать такую модальность: как и в случае наивной интерпретации, данный подход работает, если нам удаётся ограничить модели теми, в которых равенства выполняются дефинициально.

Ещё одним преимуществом модальной вычислительной реинтерпретации является следующее свойство:

Теорема 19 (Схлопывание вычислительных модальностей). *Для $MLTT\text{-}R$ справедливо следующее утверждение:*

$$\frac{\exists \kappa : \Gamma. \ell \blacksquare \vdash_{\kappa} t : T}{\forall j \geq \text{level}(T) : \Gamma. \ell \blacksquare \vdash_j \text{nf}(t) : \text{nf}(T)}$$

Прямое доказательство здесь не приводится, однако общая идея в том, что любой уровень, превышающий уровень T , может появляться лишь в каком-то редексе t или T . Тогда, если бы модальности были вычислительными, достаточно было бы использовать единственную модальность $\Box$ вместо семейства модальностей $\Box_{\ell}$: уровень типа под боксом определял бы, сколько моделей универсумов нужно предоставить.

Но, снова, всё сводится к определению типа *дефинициальных* внутренних моделей.

Реинтерпретация реинтерпретации

В разделе 2.4 мы ввели нашу модальность “реинтерпретируемых выражений” довольно жёстко ровно затем, чтобы не приходилось давать интерпретацию этой модальности внутри внутренних моделей. Опять-таки, проблема возникает с выражениями, которые пытаются интерпретировать переменные из контекста CwF ; простейший пример — $\llbracket \lambda x. \llbracket x \rrbracket_1 \rrbracket_2$. Чтобы суметь интерпретировать такое выражение, нам придётся определить *что оператор реинтерпретации значит внутри внутренней модели*. Это быстро выходит из-под контроля: вместо того чтобы просто превратить тип моделей в (рекурсивный? индуктивный?) тип, тип моделей превратился бы в тип, в котором тип одного из его полей зависит от значения всей модели. На деле такое уже случалось: Коломатская и др. [22] уже определяли тип такого рода внутри своей *теории отобразённых типов*: тип SST полу-симплициальных типов

определён коиндуктивно через значения самого себя⁴. Хотя аналогичное определение типа моделей могло бы стать интересным расширением системы типов, представленной в этой статье, на данный момент мы считаем такой подход выходящим за рамки нашей работы.

2.6 Заключение и дальнейшая работа

В ретроспективе, нам удалось определить две рефлексивные системы типов, STLC-R и MLTT-R, то есть они обладают понятием собственных “внутренних моделей”, используя которые можно перетолковывать термы самой системы внутри таких моделей. Благодаря этому такие системы типов обретают новые возможности, которые ранее задавались как отдельные встроенные особенности системы. Основные метатеоретические свойства доказаны; на будущее мы запланировали следующее:

- Возможно, стоит сделать дисциплину блокировки в разделе 2.4 более гибкой, допуская более одной блокировки в контексте. С высокой долей вероятности это потребует включить модальности в определение типа “внутренней модели” и приблизит нас к по-настоящему рефлексивной системе.
- Наша техника реинтерпретации правила конверсии в разделе 2.4 работает, но текущий подход к извлечению свидетеля равенства представляет собой *хак*. Корректное решение потребовало бы задать извлечение свидетеля как редуцируемое выражение и, возможно, ввести определённые *алгебры стратегий редукации*, чтобы сделать этот процесс более управляемым в системах, где свидетелей равенства может быть несколько.
- Подход с категориями с семействами для задания внутренних моделей здесь, в определённом смысле, представляет собой “глубокий” интерфейс: хотя все его средства корректно используются при реинтерпретации, легко добавить возможность интерпретировать выражения в произвольном контексте CwF. Это указывает на естественное расширение нашей модальности реинтерпретируемых выражений до полноценных контекстуальных типов.
- После реализации контекстуальных типов становится возможным обрабатывать “частичную” реинтерпретацию выражений: если выражение, находящееся в процессе интерпретации, не использует определённую возможность системы типов, то не должно быть необходимости указывать её в модели! Это приблизит наш язык к подходу Конала Эллиота “Компиляция в категории” [16] и к *настоящему* формальному языку для синтетической математики.
- Было бы хорошо сделать равенство в MLTT-R более экстензиональным, скорее всего, приблизив систему типов к гомотопической теории типов [35].
- В определённый момент нам придётся проявить должную тщательность: предоставить формализованные доказательства метатеоретических свойств систем и реализацию MLTT-R в практическом инструменте верификации доказательств.

⁴В более общем случае Шульман называет такие типы *модальными кодами* [32].

Заключение

Основные результаты исследования

TODO

С акцентом на изложение полученных научных результатов (а не на пересказ содержания диссертации)

Список выносимых на защиту результатов TODO

Общие выводы

TODO

(без повторения основных результатов)

Перспективы дальнейшей разработки

TODO

Основные публикации соискателя

TODO

А. Список сокращений и условных обозначений

В. Словарь терминов

С. Список литературы

- [1] Andreas Abel, Klaus Aehlig и Peter Dybjer. «Normalization by Evaluation for Martin-Löf Type Theory with One Universe». В: *Electronic Notes in Theoretical Computer Science* 173 (2007). Proceedings of the 23rd Conference on the Mathematical Foundations of Programming Semantics (MFPS XXIII), с. 17—39. ISSN: 1571-0661. DOI: <https://doi.org/10.1016/j.entcs.2007.02.025>. URL: <https://www.sciencedirect.com/science/article/pii/S1571066107000977>.
- [2] National Security Agency. *Software Memory Safety*. Апр. 2023. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF.
- [3] Thorsten Altenkirch и Nils Anders Danielsson. «Termination Checking in the Presence of Nested Inductive and Coinductive Types». В: *Partiality and Recursion in Interactive Theorem Provers, PAR@ITP 2010, Edinburgh, UK, July 15, 2010*. Под ред. Ekaterina Komendantskaya, Ana Bove и Milad Niqui. Т. 5. EPiC Series. EasyChair, 2010, с. 101—106. DOI: 10.29007/N51D.
- [4] Thorsten Altenkirch и Ondrej Rypacek. «A Syntactical Approach to Weak omega-Groupoids». В: *Computer Science Logic (CSL'12) - 26th International Workshop/21st Annual Conference of the EACSL*. Под ред. Patrick Cégielski и Arnaud Durand. Т. 16. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2012, с. 16—30. ISBN: 978-3-939897-42-2. DOI: 10.4230/LIPIcs.CSL.2012.16. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CSL.2012.16>.
- [5] Thorsten Altenkirch и др. «Internal Parametricity, without an Interval». В: *Proc. ACM Program. Lang.* 8.POPL (янв. 2024). DOI: 10.1145/3632920.
- [6] Nada Amin и Tiark Ropmf. «Collapsing towers of interpreters». В: *Proc. ACM Program. Lang.* 2.POPL (дек. 2017). DOI: 10.1145/3158140.
- [7] Jean-Philippe Bernardy, Patrik Jansson и Ross Paterson. «Parametricity and dependent types». В: *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming*. ICFP '10. Baltimore, Maryland, USA: Association for Computing Machinery, 2010, с. 345—356. ISBN: 9781605587943. DOI: 10.1145/1863543.1863592.
- [8] Marc Bezem и др. *A Note on Generalized Algebraic Theories and Categories with Families*. 2021. arXiv: 2012.08370 [math.CT]. URL: <https://arxiv.org/abs/2012.08370>.
- [9] Lars Birkedal и др. «Modal dependent type theory and dependent right adjoints». В: *Mathematical Structures in Computer Science* 30.2 (2020), с. 118—138. DOI: 10.1017/S0960129519000197.
- [10] Mario Carneiro. «The Type Theory of Lean». Дис. ... маг. Carnegie Mellon University, апр. 2019.

-
- [11] Simon Castellan, Pierre Clairambault и Peter Dybjer. *Categories with Families: Untyped, Simply Typed, and Dependently Typed*. 2020. arXiv: 1904.00827 [cs.LO]. URL: <https://arxiv.org/abs/1904.00827>.
 - [12] Dominique Clément и др. «A simple applicative language: mini-ML». В: *Proceedings of the 1986 ACM Conference on LISP and Functional Programming*. LFP '86. Cambridge, Massachusetts, USA: Association for Computing Machinery, 1986, с. 13—27. ISBN: 0897912004. DOI: 10.1145/319838.319847.
 - [13] Ranald Clouston и др. *Programming and Reasoning with Guarded Recursion for Coinductive Types*. 2015. arXiv: 1501.02925 [cs.PL].
 - [14] Thierry Coquand и Gérard Huet. «The calculus of constructions». В: *Information and Computation* 76.2 (1988), с. 95—120. ISSN: 0890-5401. DOI: [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3).
 - [15] Edsger W. Dijkstra. «The humble programmer». В: *Commun. ACM* 15.10 (окт. 1972), с. 859—866. ISSN: 0001-0782. DOI: 10.1145/355604.361591.
 - [16] Conal Elliott. «Compiling to categories». В: *Proc. ACM Program. Lang.* 1.ICFP (авг. 2017). DOI: 10.1145/3110271.
 - [17] Thiago Felicissimo. «Generic Bidirectional Typing for Dependent Type Theories». В: *ACM Trans. Program. Lang. Syst.* 47.1 (апр. 2025). ISSN: 0164-0925. DOI: 10.1145/3715095.
 - [18] Daniel Gratzer, Jonathan Sterling и Lars Birkedal. «Implementing a modal dependent type theory». В: *Proc. ACM Program. Lang.* 3.ICFP (июль 2019). DOI: 10.1145/3341711.
 - [19] Martin Hofmann и Thomas Streicher. «The groupoid model refutes uniqueness of identity proofs». В: *Proceedings of the Ninth Annual IEEE Symp. on Logic in Computer Science, LICS 1994*. Под ред. Samson Abramsky. Paris, France: IEEE Computer Society Press, июль 1994, с. 208—212.
 - [20] Kuen-Bang Hou (Favonia), Carlo Angiuli и Reed Mullanix. «An Order-Theoretic Analysis of Universe Polymorphism». В: *Proc. ACM Program. Lang.* 7.POPL (январь 2023). DOI: 10.1145/3571250.
 - [21] W. Howard. «The Formulae-as-Types Notion of Construction». В: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Под ред. J. Seldin и J. Hindley. Academic Press, 1980, с. 479—490.
 - [22] Astra Kolomatskaia и Michael Shulman. «Displayed type theory and semi-simplicial types». В: *Mathematical Structures in Computer Science* 35 (2025), e34. DOI: 10.1017/S096012952510025
 - [23] Joachim Lambek. «From λ -calculus to cartesian closed categories». В: *To H. B. Curry: Essays on Combinatory Logic, Lambda-Calculus and Formalism*. Под ред. J. P. Seldin и J. R. Hindley. London: Academic Press, 1980, с. 375—402.
 - [24] Conor McBride. *Crude but Effective Stratification*. 2011. URL: <https://mazzo.li/epilogue/index.html%3Fp=857&cpage=1.html> (дата обр. 09.07.2026).
 - [25] Conor McBride. *Crude but Effective Stratification*. 2012. URL: <https://personal.cis.strath.ac.uk/conor.mcbride/Crude.pdf> (дата обр. 09.07.2026).
 - [26] Leonardo de Moura и Sebastian Ullrich. «The Lean 4 Theorem Prover and Programming Language». В: *Automated Deduction – CADE 28*. Под ред. André Platzer и Geoff Sutcliffe. Cham: Springer International Publishing, 2021, с. 625—635. ISBN: 978-3-030-79876-5.
 - [27] Wojciech Nawrocki и др. «A Certifying Proof Assistant for Synthetic Mathematics in Lean». В: *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP '26. Rennes, France: Association for Computing Machinery, 2026, с. 88—103. ISBN: 9798400723414. DOI: 10.1145/3779031.3779087.

-
- [28] Chris Newcombe и др. «How Amazon web services uses formal methods». В: *Commun. ACM* 58.4 (март 2015), с. 66—73. ISSN: 0001-0782. DOI: 10.1145/2699417.
 - [29] Ulf Norell. «Towards a practical programming language based on dependent type theory». Дис. . . . док. SE-412 96 Göteborg, Sweden: Department of Computer Science и Engineering, Chalmers University of Technology, сент. 2007.
 - [30] John C Reynolds. «Types, abstraction and parametric polymorphism». В: *Information Processing 83*. Под ред. R. E. A. Mason. North-Holland. 1983, с. 513—523.
 - [31] Gregor Richards. *Proof of the Church-Rosser Theorem*. 2025. URL: <https://student.cs.uwaterloo.ca/~cs442/W25/extras/c-r-thm-proof.pdf> (дата обр. 19.02.2026).
 - [32] Michael Shulman. *Modal datatypes and codatatypes*. 2026. URL: <https://gwaithimirdain.github.io/2026/06/02/modal-data-and-codata.html> (дата обр. 09.07.2026).
 - [33] Peiyang Song, Kaiyu Yang и Anima Anandkumar. *Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean*. 2025. arXiv: 2404.12534 [cs.AI].
 - [34] The Coq Development Team. *The Coq Proof Assistant*. Вер. 8.20. Сент. 2024. DOI: 10.5281/zenodo.14542673.
 - [35] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: <https://homotopytypetheory.org/book>, 2013.
 - [36] Andrea Vezzosi, Anders Mörtberg и Andreas Abel. «Cubical agda: a dependently typed programming language with univalence and higher inductive types». В: *Proc. ACM Program. Lang.* 3.ICFP (июль 2019). DOI: 10.1145/3341691.
 - [37] Philip Wadler. «Propositions as types». В: *Commun. ACM* 58.12 (нояб. 2015), с. 75—84. ISSN: 0001-0782. DOI: 10.1145/2699407.
 - [38] Б. Пирс. *Типы в языках программирования: пер. с англ.* Лямбда пресс, 2012. ISBN: 9785791300829.

D. Список иллюстрированного материала

Е. Приложения